

PC 端 Web SDK React 接口说明文档 V4.0.6

文档修改记录

序号	版本号	修改内容	修改者	修改日期
1	v3.1.2	文档建立	周功成	2021/4/7
2	v3.1.4	更新 3.1.4 内容	石坤	2021/11/15
3	v3.1.8	- 兼容新机型 - 修复旋转后裁切异常 bug	石坤	2022/8/8
4	v3.1.8	更新接口封装	张彬	2023/6/7
5	v3.2.1	- 支持 K3 及 K3W 机型 - 增加 WIFI 相关接口	张彬	2023/10/12
6	v3.2.2	- 支持 M2 机型 - 增加 M2 相关说明	张彬	2023/10/30
7	v3.2.5	- 支持 B3S_P 机型 - 支持 B21S 机型 - 支持 B31 机型 - 更新图像库	张彬	2024/9/12
8	v4.0.3	- 支持 M3、K2、B21Pro 系列机型 - 完善错误码 - 新增绘制带 logo 二维码接口	张彬	2025/4/29

		提高 Websocket 通讯速度 5.Demo 支持黑标间隙纸		
9	v4.0.6	- 新增 closePrinter 接口 - 修复 Wifi 搜索接口 BUG - 修复历史遗留 WIFI 连接 BUG	张彬	2025/9/13
10	v4.0.6	修复文本换行模式BUG	张彬	2025/11/20

DEMO 目录结构

代码块

```
1  pc-react/
2  |— README.md           # 项目说明文档
3  |— eslint.config.js    # ESLint配置文件
4  |— index.html          # 入口HTML文件
5  |— package.json        # 项目依赖和脚本配置
6  |— package-lock.json   # 依赖版本锁定文件
7  |— public/            # 静态资源目录
8  |— src/               # 源代码目录
9  |   |— App.jsx         # 应用主组件
10 |   |— Home.jsx        # 主页面组件
11 |   |— HomeLogic.ts    # 主页业务逻辑类
12 |   |— Print.ts        # 打印功能核心类
13 |   |— PrintElementFactory.ts # 打印元素工厂类
14 |   |— Socket.ts       # Socket通信类
15 |   |— assets/         # 资源文件目录
16 |   |— home.css        # 主页样式文件
17 |   |— index.css       # 全局样式文件
18 |   |— main.jsx        # 应用入口文件
19 |   |— printData/      # 打印数据目录
20 |       |— Barcode.ts  # 条码打印数据
21 |       |— Batch.ts    # 批量打印数据
22 |       |— Combination.ts # 组合打印数据
23 |       |— Graph.ts    # 图形打印数据
24 |       |— Img.ts       # 图片打印数据
25 |       |— Line.ts      # 线条打印数据
26 |       |— QrCode.ts    # 二维码打印数据
```

27 | └─ Text.ts # 文本打印数据

28 | └─ tsconfig.json # TypeScript配置文件

29 | └─ vite.config.js # Vite构建配置文件

产品目的

JCAPI 接口为调用者提供易用的方法完成标签绘图、打印操作。本接口中提供了标贴的绘制方法，包括：文字、一维码、二维码，图形、线条、图像绘制，同时还能进行绘制对象的旋转，调用者还可以调用方法获得绘制完成的标签图片用于标签预览，打印。方便用户在二次开发中调用接口，缩短开发周期，加快开发

打印机支持

支持打印机型号
B1
B203
B21 /B21_Pro/B21S
B3S / B3S_P
B31
B4
B11
K2
K3/K3W
B50/B50W
B32/Z401/B32R
M2
M3

准备工作

- 安装精臣打印服务（jcPrinterSdk.exe）

- 前置：关闭杀毒软件（如 360，易误报）
 - 关键：**必须默认路径安装（C 盘）**
 - 注意：勿禁用服务开机启动
- 安装对应机型驱动

机型系列	系统要求
B50/B11	Win7/10/11 均需装驱动
其他机型	Win10/11 无需装，仅 Win7 需装

- 设备连接（2 种方式，不支持蓝牙）
 - USB 连接
 - 系统：仅支持 Windows
 - 驱动：可能需装（参考第 2 步）
 - 特别：****不支持驱动打印****（已用驱动打印需下载专用驱动）
 - WIFI 连接
 - 机型：**仅支持 K3W 机型**
 - 系统：仅支持 Windows
 - 驱动：无需安装

一、初始化及打印调用流程、打印流程

1.1 初始化流程

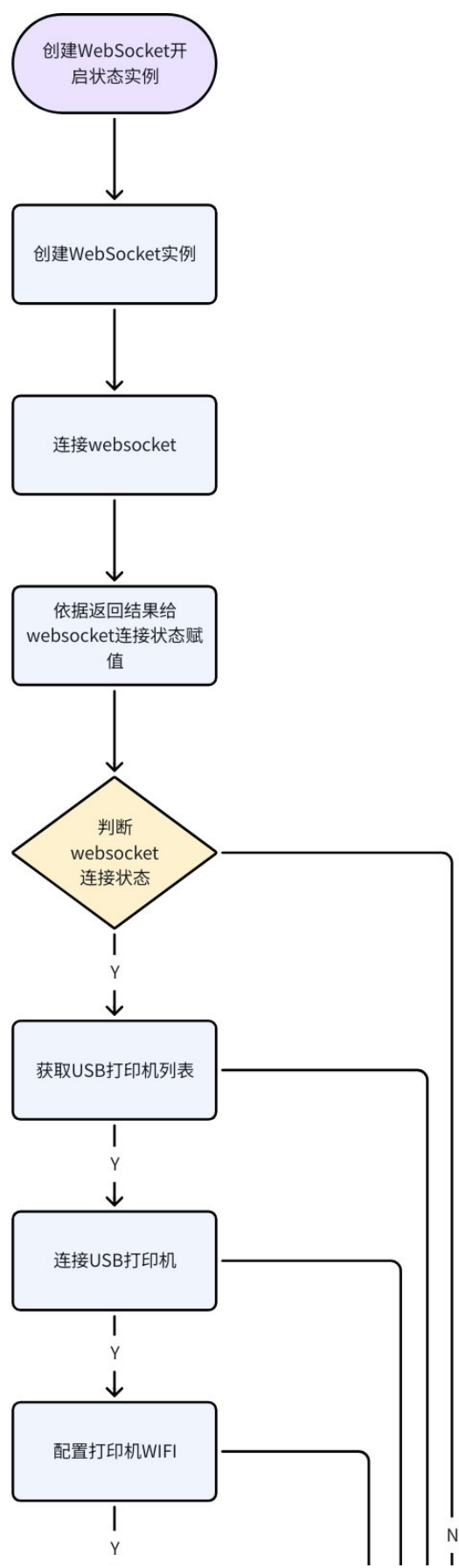
1.1.1 USB 打印初始化流程

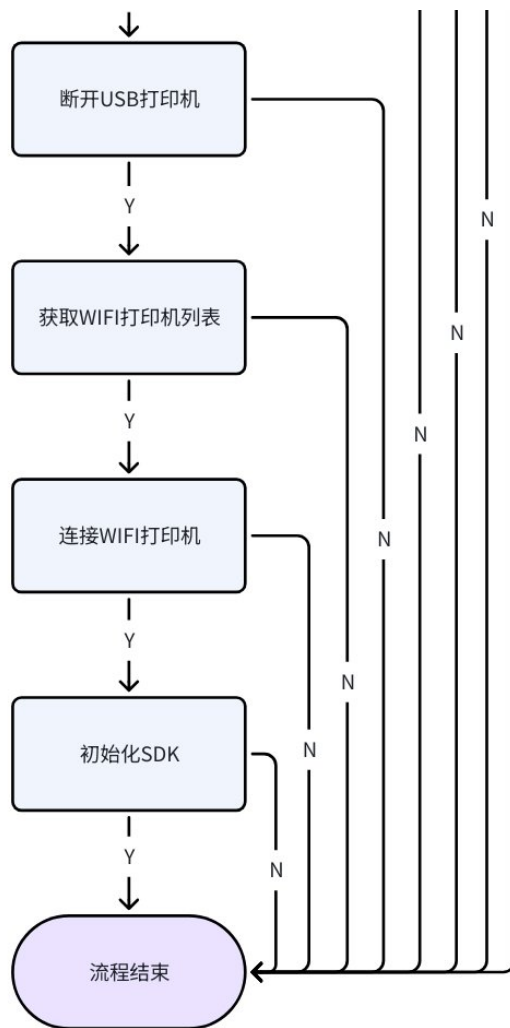
- WebSocket 建议页面加载时进行初始化，在 WebSocket 初始成功后回调中进行获取打印机、选择打印机、初始化 SDK 等操作
- 因为所有接口均为异步操作，调用下一接口需要验证当前接口结果后再执行下一接口
- 记录打印机列表获取状态、连接状态、初始化状态，打印机需要检查对应的状态

1.1.2 WIFI 打印初始化流程

- WebSocket 建议页面加载时进行初始化，在 WebSocket 初始成功后回调中进行获取打印机、选择打印机、初始化 SDK 等操作

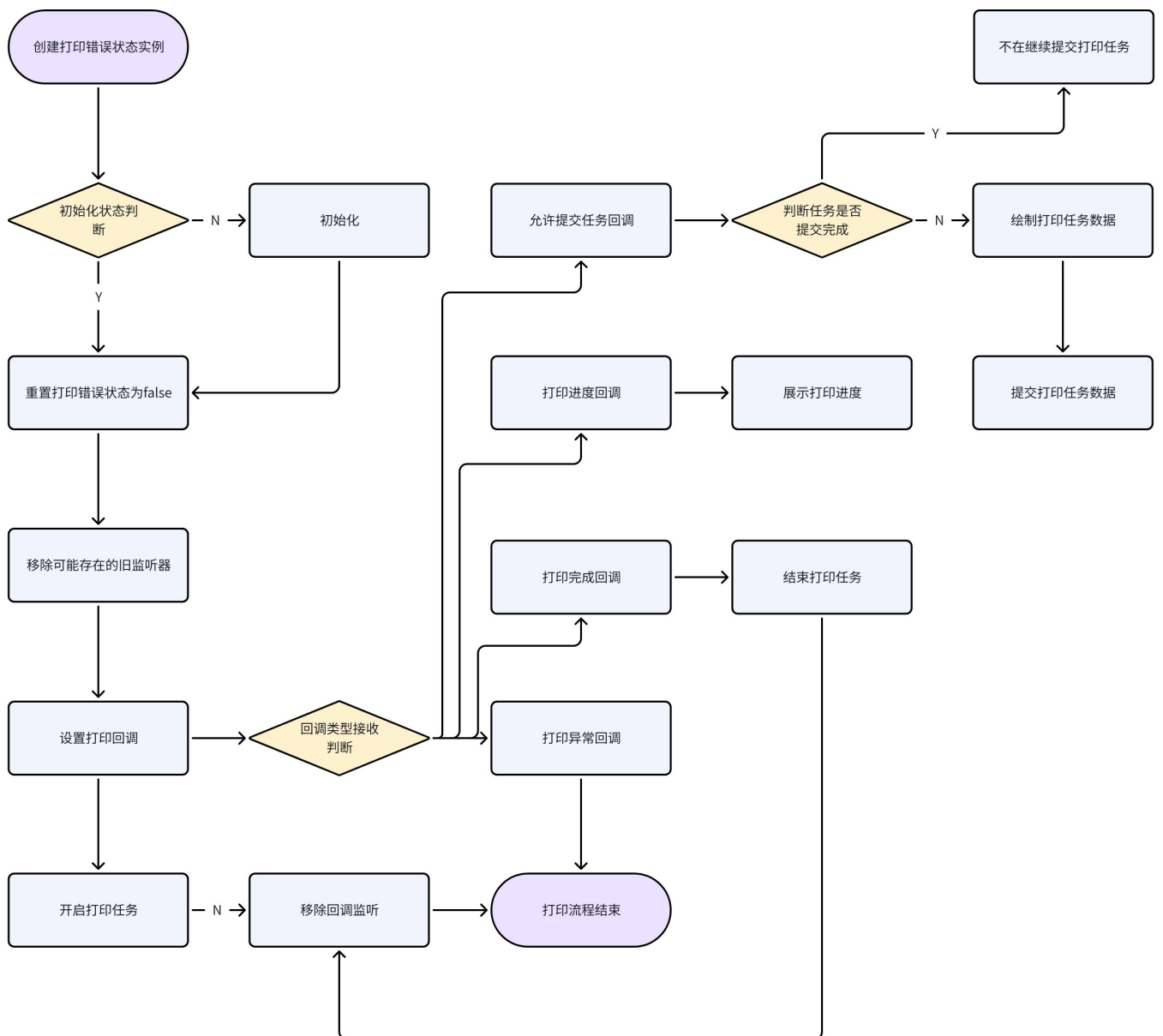
- 因为所有接口均为异步操作，调用下一接口需要验证当前接口结果后再执行下一接口
- 记录打印机列表获取状态、连接状态、初始化状态，打印机需要检查对应的状态
- 打印机 WIFI 配置成功后，后续直接搜索连接，无需多次进行配置（省略 USB 打印机获取、打印机连接、打印机网络配置）





1.2 打印流程

- 打印前建议判断 WebSocket 是否初始化成功、SDK 是否初始化成功（包含初始化 SDK，获取打印机、选择打印机三个流程）
- 因为所有接口均为异步操作，除 WebSocket 初始化调用是在单独的回调接口中判断是否初始化成功外，其他接口应通过添加 await 关键字调用方法后，等待方法返回结果，解析返回的结果数据后进行判断再进行下一接口调用
- 打印回调监听会有多种回调，包含异常取消、页码回调，可参考流程图及 DEMO 进行处理



二、页面初始化相关接口

2.1 初始化打印服务及接口实例（包含打印机状态回调）

代码块

```

1  export default class Socket {
2    /**
3     * 打开 WebSocket 连接并返回一个解析为 WebSocket 实例的 Promise。
4     *
5     * @param {function} openChange - WebSocket 连接打开时要调用的回调函数。
6     * @param {function} onMessageCallback - 接收到消息时要调用的回调函数。
7     * @return {Promise} 一个解析为 WebSocket 实例的 Promise。

```



```

8      */
9      public open(openChange?: (isOpen: boolean) => void, onMessageCallback?:
    (msg: any) => void): Promise<{ e: Event, ws: Socket }> {}
10  }
11
12
13  export default class NMPrintSocket {
14      constructor(printSocketData: Socket) {
15          this.printSocket = printSocketData;
16      }
17  }

```

代码块

```

1  export class HomeLogic {
2      // WebSocket实例
3      public socketData: Socket | null = null;
4
5      // 打印Socket实例
6      public nMPrintSocket: NMPrintSocket | null = null;
7      public initialize(): void {
8          // 创建Socket实例
9          this.socketData = new Socket();
10
11          // 打开WebSocket连接
12          this.socketData.open(
13              // 连接状态回调
14              async (openBool) => {
15                  console.log("WebSocket连接状态:", openBool);
16                  // 更新套接字连接状态
17                  this.printSocketOpen = openBool;
18                  this._updateReactState();
19              },
20              // 消息处理回调
21              (msg: any) => {
22                  // 处理设备状态回调
23                  if (msg.resultAck.callback !== undefined) {
24                      const callbackName = msg.resultAck.callback.name;
25                      const msgInfo = msg.resultAck.info;
26                      if (callbackName === "onCoverStatusChange") {
27                          console.log("盒盖状态", msgInfo.capStatus);
28                      } else if (callbackName === "onElectricityChange") {
29                          console.log("电池电量等级", msgInfo.power);
30                      }
31                  }
32              }

```

```

33         );
34
35         // 创建打印Socket实例
36         this.nMPrintSocket = new NMPrintSocket(this.socketData);
37         this._updateReactState();
38     }
39 }

```

2.2 初始化 SDK initSdk

代码块

```

1  export default class NMPrintSocket {
2      /**
3       * 初始化SDK，在打印服务连接成功后调用此接口。
4       * 在调用SDK的绘制接口之前，必须先调用此接口。
5       *
6       * @param {object} json - 包含必要参数的JSON对象,格式如下:
7       * {
8       *   "fontDir": string, //字体文件目录，默认为"", 暂不生效
9       * }
10      *
11      * @return {Promise} 返回一个 Promise，解析为初始化SDK的结果
12      */
13      public initSdk(json: { fontDir: string }): Promise<any>
14  }

```

代码块

```

1  //初始化SDK参数JSON
2  {
3      "fontDir": ""
4  }
5  //初始化成功返回JSON
6  {
7      "apiName": "initSdk",
8      "resultAck": {
9          "errorCode": 0,
10         "info": "initSdkApi ok!",
11         "result": 0
12     }
13 }
14
15 /**

```

```

16  * 初始化SDK
17  * 通过API调用初始化SDK
18  */
19  public async init(): Promise<void> {
20      if (!this.printSocketOpen || !this.nMPrintSocket) {
21          return alert("打印服务未开启");
22      }
23      if (!this.onlineUsbBool && !this.onlineWifiBool) {
24          return alert("打印机未连接");
25      }
26      try {
27          const initRes = await this.nMPrintSocket.initSdk({ fontDir: "" });
28          const result = JSON.parse(initRes.resultAck.errorCode);
29          if (result === 0) {
30              this.initBool = true;
31              console.log("SDK初始化成功");
32          } else {
33              this.initBool = false;
34              console.log("SDK初始化失败");
35              alert("SDK初始化失败");
36          }
37      } catch (err) {
38          console.error(err);
39          this.initBool = false;
40          alert("SDK初始化异常");
41      }
42      this._updateReactState();
43  }

```

2.3 获取 USB 打印机列表 getAllPrinters

代码块

```

1  export default class NMPrintSocket {
2      /**
3       * 获取所有当前PC上通过USB连接的精臣打印机
4       *
5       * @return {Promise} 返回一个Promise，解析为打印机列表。
6       *
7       * @description
8       * 需要在打印服务连接成功后调用此函数。
9       */
10     public getAllPrinters(): Promise<any>
11 }

```

代码块/返回结果

```
2  {
3      "apiName": "getAllPrinters",
4      "resultAck": {
5          "errorCode": 0,
6          "info": "{\\\"e623012991\\\":\\\"31\\\"}", //打印机名称及类型
7          "result": "true"
8      }
9  }
10
11 /**
12  * 获取所有USB打印机
13  * 通过API调用获取所有已连接的USB打印机
14  */
15 public async getPrinters(): Promise<void> {
16     if (!this.printSocketOpen || !this.nMPrintSocket) {
17         // alert("打印服务未开启");
18         console.log("打印服务未开启");
19         return;
20     }
21     console.log("开始获取打印机");
22     try {
23         const allPrintersRes = await this.nMPrintSocket.getAllPrinters();
24         console.log(allPrintersRes, "allPrintersRes");
25         if (allPrintersRes.resultAck.errorCode === 0) {
26             const allPrinters = JSON.parse(allPrintersRes.resultAck.info);
27             this.usbPrinters = { ...allPrinters };
28             this.usbSelectPrinter = Object.keys(this.usbPrinters)[0] || "";
29             console.log("printers", this.usbPrinters);
30         } else {
31             this.usbPrinters = {};
32             this.usbSelectPrinter = "";
33             alert("没有在线的USB打印机");
34         }
35     } catch (err) {
36         console.error(err);
37         this.usbPrinters = {};
38         this.usbSelectPrinter = "";
39         alert("获取USB打印机列表失败");
40     }
41     this._updateReactState();
42 }
```

2.4 获取 WIFI 连接的打印机列表 scanWifiPrinter

代码块

```

1  export default class NMPrintSocket {
2      /**
3       * 搜索Wifi打印机
4       *
5       *
6       * @return {Promise} 返回一个 Promise, 解析为打印机Wifi配置信息
7       *
8       * @description
9       * 需要在打印服务连接成功后调用此函数。
10      */
11      public scanWifiPrinter(): Promise<any>
12  }

```

代码块

```

1  //返回结果
2  {
3      "apiName": "scanWifiPrinter",
4      "resultAck": {
5          "errorCode": 0,
6          "info": "[{
7      "deviceName": "K3W-E828013369",
8          "IP": "192.168.1.10",
9          "tcpPort": "9200",
10         "availableClient": "0"
11     }]",
12     "result": "true"
13     }
14 }
15
16
17 /**
18 * 扫描所有Wifi打印机
19 * 通过API调用扫描所有可用的Wifi打印机
20 */
21 public async scanWifiPrinters(): Promise<void> {
22     if (!this.printSocketOpen || !this.nMPrintSocket) {
23         alert("打印服务未开启");
24         return;
25     }
26     try {
27         const allPrintersRes = await this.nMPrintSocket.scanWifiPrinter();
28         console.log("allPrintersRes", allPrintersRes);
29         const errorCode = allPrintersRes.resultAck.errorCode;
30         if (errorCode === 0) {

```

```

31         const allPrinters: WifiPrinterInfo[] =
allPrintersRes.resultAck.info;
32         this.wifiPrinters = {};
33         allPrinters.forEach((item) => {
34             this.wifiPrinters[item.deviceName] = item.tcpPort.toString();
35         });
36         console.log("wifiPrinters", this.wifiPrinters);
37         this.wifiSelectPrinter = Object.keys(this.wifiPrinters)[0] || "";
38         console.log("wifiSelectPrinter", this.wifiSelectPrinter);
39     } else {
40         this.wifiPrinters = {};
41         this.wifiSelectPrinter = "";
42         alert("没有在线的Wifi打印机");
43     }
44 } catch (err) {
45     console.error(err);
46     this.wifiPrinters = {};
47     this.wifiSelectPrinter = "";
48     alert("扫描Wifi打印机列表失败");
49 }
50 this._updateReactState();
51 }

```

2.5 连接 USB 打印机 selectPrinter

代码块

```

1  export default class NMPrintSocket {
2      /**
3       * 选择并打开需要使用的打印机名称，及端口号
4       *
5       * @param {string} printerName - 打印机名称。
6       * @param {number} port - 连接端口。
7       * @return {Promise} 返回一个Promise，解析为连接结果
8       *
9       * @description
10      * 需要在打印服务连接成功后调用此函数，建议在getAllPrinters调用成功后调用该接口，以
      保证传入的打印机名称和端口的打印机状态正常。。
11      */
12      public selectPrinter(printerName: string, port: number): Promise<any>
13  }

```

代码块

```

1  //返回数据示例
2  {

```

```

3         "apiName": "selectPrinter",
4         "resultAck": {
5             "callback": {
6                 "name": "onConnectSuccess",
7                 "printerName": "e623012991"
8             },
9             "errorCode": 0,
10            "info": "select printer ok!",
11            "result": true
12        }
13    }
14
15    /**
16     * 连接选中的USB打印机
17     * 通过API调用连接用户选择的USB打印机
18     */
19    public async selectOnLineUsbPrinter(): Promise<void> {
20        if (!this.printSocketOpen || !this.nMPrintSocket) {
21            return alert("打印服务未开启");
22        }
23        if (!this.usbSelectPrinter) {
24            alert("请先选择一个USB打印机");
25            return;
26        }
27        try {
28            console.log("this.usbSelectPrinter", this.usbSelectPrinter);
29            console.log("this.usbPrinters[this.usbSelectPrinter]",
30            this.usbPrinters[this.usbSelectPrinter]);
31            const usbConnectRes = await
32            this.nMPrintSocket.selectPrinter(this.usbSelectPrinter,
33            parseInt(this.usbPrinters[this.usbSelectPrinter]));
34            const result = JSON.parse(usbConnectRes.resultAck.errorCode);
35            console.log("result", result);
36            if (result === 0) {
37                console.log("USB打印机连接成功");
38                this.onlineUsbBool = true;
39                this.onlineWifiBool = false;
40            } else {
41                console.log("USB打印机连接失败");
42                this.onlineUsbBool = false;
43                alert("USB打印机连接失败");
44            }
45            console.log("usbConnectRes", usbConnectRes);
46        } catch (err) {
47            console.error(err);
48            this.onlineUsbBool = false;
49            alert("连接USB打印机异常");
50        }
51    }

```

```
47     }
48     this._updateReactState();
49 }
```

2.6 连接 WIFI 打印机列表中的打印机 connectWifiPrinter

代码块

```
1  export default class NMPrintSocket {
2      /**
3       * 发送消息以选择打印机。
4       *
5       * @param {string} printerName - 打印机名称。
6       * @param {number} tcpPort - 端口号。
7       * @return {Promise} 返回连接结果
8       *
9       * @description
10      * 需要在打印服务连接成功后调用此函数，建议在scanWifiPrinter调用成功的回调接口中调用该接口，保证传入的打印机名称和端口的打印机状态正常。
11      * 注意：此函数仅能连接 WIFI 打印机列表中的打印机。
12      */
13      public connectWifiPrinter(printerName: string, tcpPort: number):
14      Promise<any>
15  }
```

代码块

```
1  //示例返回成功数据
2  {
3      "apiName": "selectPrinter",
4      "resultAck": {
5          "callback": {
6              "name": "onConnectSuccess",
7              "printerName": "e623012991"
8          },
9          "errorCode": 0,
10         "info": "select printer ok!",
11         "result": true
12     }
13 }
14 //示例返回失败数据
15 {
16     "apiName": "connectWifiPrinter",
17     "resultAck": {
18         "callback": {
19             "name": "onDisConnect",
```



```
20     "printerName": "K3_W-F612010061"
21   },
22   "errorCode": 0,
23   "info": "success",
24   "result": false
25 }
26 }
27
28 /**
29  * 连接选中的Wifi打印机
30  * 通过API调用连接用户选择的Wifi打印机
31  */
32 public async selectOnlineWifiPrinter(): Promise<void> {
33   if (!this.printSocketOpen || !this.nMPrintSocket) {
34     return alert("打印服务未开启");
35   }
36   if (!this.wifiSelectPrinter || !this.wifiPrinters[this.wifiSelectPrinter])
37 {
38   alert("请先选择一个有效的Wifi打印机");
39   return;
40 }
41   try {
42     const wifiConnectRes = await this.nMPrintSocket.connectWifiPrinter(
43       this.wifiSelectPrinter,
44       parseInt(this.wifiPrinters[this.wifiSelectPrinter])
45     );
46     const result = JSON.parse(wifiConnectRes.resultAck.errorCode);
47     if (result) {
48       console.log("Wifi打印机连接成功");
49       this.onlineWifiBool = true;
50       this.onlineUsbBool = false;
51     } else {
52       console.log("Wifi打印机连接失败");
53       this.onlineWifiBool = false;
54       alert("Wifi打印机连接失败");
55     }
56     console.log("wifiConnectRes", wifiConnectRes);
57   } catch (err) {
58     console.error(err);
59     this.onlineWifiBool = false;
60     alert("连接Wifi打印机异常");
61   }
62   this._updateReactState();
63 }
```

2.7 断开打印机连接closePrinter

代码块

```
1  export default class NMPrintSocket {
2      /**
3       * 断开打印机连接。
4       *
5       * @return {Promise} 返回一个Promise, 解析为关闭结果
6       */
7      public closePrinter(): Promise<any>
8  }
```

2.8 配置打印机的 WIFI 信息 configurationWifi

代码块

```
1  export default class NMPrintSocket {
2      /**
3       * 配置打印机的Wifi网络
4       *
5       * @param {string} wifiName - wifi网络的名称。
6       * @param {string} wifiPassword - wifi网络的密码。
7       * @return {Promise} 返回一个 Promise, 解析为打印机Wifi配置结果
8       *
9       * @description
10      * 注意:仅支持2.4G频段网络, 且需要在连接成功后配置。首次配置建议在USB连接成功后配置
11      */
12      public configurationWifi(wifiName: string, wifiPassword: string):
13          Promise<any>
14  }
```

代码块

```
1  //示例返回数据
2  {
3      "apiName":"configurationWifi",
4      "resultAck":{
5          "errorCode":0,
6          "info":"configuration wifi printer ok!",
7          "result":true
8      }
9  }
```

```

11  /**
12   * 配置打印机的Wifi网络
13   * 通过API调用配置打印机的Wifi网络
14   */
15  public async setWifiConfiguration(): Promise<void> {
16      if (!this.printSocketOpen || !this.nMPrintSocket) {
17          return alert("打印服务未开启");
18      }
19      try {
20          if (this.wifiName.trim() !== "") {
21              const wifiConfigurationResult = await
this.nMPrintSocket.configurationWifi(
22                  this.wifiName.trim(),
23                  this.wifiPassword.trim()
24              );
25              console.log("wifiConfigurationResult", wifiConfigurationResult);
26              const errorCode = wifiConfigurationResult.resultAck.errorCode;
27              if (errorCode === 0) {
28                  alert("网络配置成功，请断开USB线缆后使用WIFI搜索连接打印机（PC需要和打
打印机在同一网络）");
29              } else {
30                  alert("网络配置失败，错误码：" + errorCode);
31              }
32          } else {
33              alert("wifi名称不得为空");
34          }
35      } catch (err) {
36          console.error(err);
37          alert("配置Wifi网络异常");
38      }
39  }

```

2.9 获取打印机的 WIFI 相关配置 getWifiConfiguration

代码块

```

1  export default class NMPrintSocket {
2      /**
3       * 获取打印机的wifi配置。
4       *
5       * @return {Promise} 返回一个 Promise，解析为打印机Wifi配置信息
6       */
7      public getWifiConfiguration(): Promise<any>
8  }

```


三、绘制打印数据相关接口

3.1 创建画板 InitDrawingBoard

代码块

```

1  export default class NMPrintSocket {
2      /**
3       * 初始化绘制画板
4       *
5       * @param {Object} json - 包含初始化绘制画板所需数据的JSON对象。格式如下：
6       * {
7       *   "width": number, // 画板的宽度，单位为mm
8       *   "height": number, // 画板的高度，单位为mm
9       *   "rotate": number, // 画板的旋转角度，仅支持0、90、180、270
10      *   "path": string, // 字体文件的路径，默认为"", 暂不生效
11      *   "verticalShift": number, // 垂直偏移量，暂不生效
12      *   "HorizontalShift": number // 水平偏移量，暂不生效
13      * }
14      * @return {Promise} 返回一个 Promise，解析为初始化绘制画板的结果
15      *
16      * @description
17      * 增加接口说明：
18      * 1. 在调用绘制接口之前，必须先初始化SDK。
19      * 2. 绘制元素前，必须先初始化画板，否则会引起崩溃！
20      * 3. 初始化画板时会清空画板上次绘制的内容！
21      */
22      public InitDrawingBoard(json: String): Promise<any>
23  }
```

代码块

```

1  {
2      "apiName": "InitDrawingBoard",
3      "resultAck": {
4          "errorCode": 0,
5          "info": "init draw board success!",
6          "result": 0
7      }
8  }
```

```

9
10 /**
11  * 初始化打印画布
12  * 通过API调用初始化打印画布
13  * @param params - 画布初始化参数
14  * @returns 是否初始化成功
15  */
16 private async initCanvas(params: String): Promise<boolean> {
17     if (!this.printSocketOpen || !this.nMPrintSocket || !this.initBool) {
18         return false;
19     }
20     console.log("初始化打印画布");
21     try {
22         const res = await this.nMPrintSocket.InitDrawingBoard(params);
23         return res.resultAck.errorCode === 0;
24     } catch (err) {
25         console.error('画布初始化错误:', err);
26         return false;
27     }
28 }
29 }

```

3.2 绘制文本 DrawLabelText

代码块

```

1  export default class NMPrintSocket {
2      /**
3       * 绘制标签文本。
4       * @param {object} json - 包含标签文本信息的JSON对象。
5       *   JSON格式要求如下：
6       *   - x: x轴坐标, 单位mm
7       *   - y: y轴坐标, 单位mm
8       *   - height: 文本高度, 单位mm
9       *   - width: 文本宽度, 单位mm
10      *   - value: 文本内容
11      *   - fontFamily: 字体名称, 暂不生效, 使用默认字体鸿蒙字体
12      *   - rotate: 旋转角度, 0, 90, 180, 270
13      *   - fontSize: 字号, 单位mm
14      *   - textAlignHorizonral: 水平对齐方式: 0:左对齐 1:居中对齐 2:右对齐
15      *   - textAlignVertical: 垂直对齐方式: 0:顶对齐 1:垂直居中 2:底对齐
16      *   - letterSpacing: 字母之间的标准间隔, 单位mm
17      *   - lineSpacing: 行间距 (倍距) , 默认1
18      *   - lineMode: 1:宽高固定, 内容大小自适应, 预设宽高过大时字号放大, 预设宽高过小时
19      *       字号缩小,
20      *       保证内容占据满预设宽高 (字号/字符间距/行间距 按比例缩放)

```

```

20      *      2:宽度固定, 高度自适应
21      *      4:宽高固定,超出内容直裁切
22      *      6:宽高固定, 内容超过预设的文本宽高自动缩放
23      *      建议设置为6
24      *      - fontStyle: 字体样式[加粗, 斜体, 下划线, 删除下划线 (预留)]
25      *
26      * @return {Promise} 返回一个 Promise, 解析为绘制标签文本的结果
27      * @description 绘制标签文本前必须先初始化画板
28      */
29      public DrawLabelText(json: String): Promise<any>
30  }

```

代码块

```

1  //返回数据示例
2  {
3      "apiName": "DrawLableText",
4      "resultAck": {
5          "errorCode": 0,
6          "info": "draw bar code success!",//此处返回信息有误, 下个版本修复
7          "result": 0
8      }
9  }
10
11 /**
12  * 打印元素处理方法
13  * 支持多种打印元素类型, 按顺序执行绘制操作
14  * @param elements - 打印元素数组
15  * @returns 是否全部元素处理成功
16  */
17 private async processPrintElements(elements: any[]): Promise<boolean> {
18     if (!this.nMPrintSocket) return false;
19     console.log("elements", elements);
20     for (const element of elements) {
21         let res;
22         switch (element.type) {
23             case "text": // 文本打印
24                 res = await this.nMPrintSocket.DrawLabelText(element.json);
25                 break;
26             case "qrCode": // 二维码打印
27                 res = await this.nMPrintSocket.DrawLabelQrCode(element.json);
28                 break;
29             case "barCode": // 条形码打印
30                 res = await this.nMPrintSocket.DrawLabelBarCode(element.json);
31                 break;
32             case "line": // 线条绘制

```

```

33         res = await this.nMPrintSocket.DrawLabelLine(element.json);
34         break;
35     case "graph": // 图形绘制
36         res = await this.nMPrintSocket.DrawLabelGraph(element.json);
37         break;
38     case "image": // 图像打印
39         res = await this.nMPrintSocket.DrawLabelImage(element.json);
40         break;
41     default:
42         console.error("Unsupported element type:", element.type);
43         return false;
44     }
45     if (parseInt(JSON.parse(res.resultAck.errorCode)) !== 0) {
46         console.error(`Failed to draw ${element.type}:`, res);
47         return false;
48     }
49 }
50 return true;
51 }

```

3.3 一维码绘制 DrawLabelBarCode

代码块

```

1  export default class NMPrintSocket {
2      /**
3       * 绘制一维码条形码。
4       *
5       * @param {Object} json - 包含一维码条形码信息的JSON对象。格式如下:
6       * {
7       *   "x": number, // x轴坐标, 单位mm
8       *   "y": number, // y轴坐标, 单位mm
9       *   "height": number, // 一维码宽度, 单位mm
10      *   "width": number, // 一维码高度, 单位mm (包含文本高度)
11      *   "value": string, // 一维码内容
12      *   "codeType": number, // 条码类型:
13      *                       // 20: CODE128
14      *                       // 21: UPC-A
15      *                       // 22: UPC-E
16      *                       // 23: EAN8
17      *                       // 24: EAN13
18      *                       // 25: CODE93
19      *                       // 26: CODE39
20      *                       // 27: CODEBAR
21      *                       // 28: ITF25
22      *   "rotate": number, // 旋转角度, 0, 90, 180, 270

```



```

23      *   "fontSize": number, // 文本字号, 单位mm, 字号为0则文本不显示
24      *   "textHeight": number, // 文本高度, 单位mm, 高度为0则文本不显示
25      *   "textPosition": number // 一维码文字识别码显示位置:
26      *                               // 0: 下方显示
27      *                               // 1: 上方显示
28      *                               // 2: 不显示
29      * }
30      *
31      * @return {Promise} 返回一个 Promise, 解析为绘制一维码条形码的结果
32      *
33      * @description
34      * 1. 绘制元素前, 必须先初始化画板
35      */
36      public DrawLabelBarCode(json: String): Promise<any>
37  }

```

代码块

```

1  //返回数据示例
2  {
3      "apiName": "DrawLableBarCode",
4      "resultAck": {
5          "errorCode": 0,
6          "info": "draw bar code success!",
7          "result": 0
8      }
9  }
10
11 /**
12  * 打印元素处理方法
13  * 支持多种打印元素类型, 按顺序执行绘制操作
14  * @param elements - 打印元素数组
15  * @returns 是否全部元素处理成功
16  */
17 private async processPrintElements(elements: any[]): Promise<boolean> {
18     if (!this.nMPrintSocket) return false;
19     console.log("elements", elements);
20     for (const element of elements) {
21         let res;
22         switch (element.type) {
23             case "text": // 文本打印
24                 res = await this.nMPrintSocket.DrawLabelText(element.json);
25                 break;
26             case "qrCode": // 二维码打印
27                 res = await this.nMPrintSocket.DrawLabelQrCode(element.json);
28                 break;

```

```

29         case "barCode": // 条形码打印
30             res = await this.nMPrintSocket.DrawLabelBarCode(element.json);
31             break;
32         case "line": // 线条绘制
33             res = await this.nMPrintSocket.DrawLabelLine(element.json);
34             break;
35         case "graph": // 图形绘制
36             res = await this.nMPrintSocket.DrawLabelGraph(element.json);
37             break;
38         case "image": // 图像打印
39             res = await this.nMPrintSocket.DrawLabelImage(element.json);
40             break;
41         default:
42             console.error("Unsupported element type:", element.type);
43             return false;
44     }
45     if (parseInt(JSON.parse(res.resultAck.errorCode)) !== 0) {
46         console.error(`Failed to draw ${element.type}:`, res);
47         return false;
48     }
49 }
50 return true;
51 }

```

3.4.1 二维码绘制 DrawLabelQrCode

代码块

```

1  export default class NMPrintSocket {
2      /**
3       * 绘制二维码。
4       *
5       * @param {Object} json - 包含二维码信息的JSON对象。格式如下：
6       * {
7       *     "x": number, // x轴坐标, 单位mm
8       *     "y": number, // y轴坐标, 单位mm
9       *     "height": number, // 二维码高度, 默认宽高一致
10      *     "width": number, // 二维码宽度, 单位mm
11      *     "value": string, // 二维码内容
12      *     "codeType": number, // 条码类型:
13      *                          // 31: QR_CODE
14      *                          // 32: PDF417
15      *                          // 33: DATA_MATRIX
16      *                          // 34: AZTEC
17      *     "rotate": number, // 旋转角度, 仅支持0、90、180、270
18      * }

```

```

19      *
20      * @return {Promise} 返回一个 Promise，解析为绘制二维码的结果
21      *
22      * @description
23      * 1. 绘制元素前，必须先初始化画板
24      */
25      public DrawLabelQrCode(json: String): Promise<any>
26  }

```

代码块

```

1  //返回数据示例
2  {
3      "apiName": "DrawLableQrCode",
4      "resultAck": {
5          "errorCode": 0,
6          "info": "draw qr code success!",
7          "result": 0
8      }
9  }
10
11 /**
12  * 打印元素处理方法
13  * 支持多种打印元素类型，按顺序执行绘制操作
14  * @param elements - 打印元素数组
15  * @returns 是否全部元素处理成功
16  */
17 private async processPrintElements(elements: any[]): Promise<boolean> {
18     if (!this.nMPrintSocket) return false;
19     console.log("elements", elements);
20     for (const element of elements) {
21         let res;
22         switch (element.type) {
23             case "text": // 文本打印
24                 res = await this.nMPrintSocket.DrawLabelText(element.json);
25                 break;
26             case "qrCode": // 二维码打印
27                 res = await this.nMPrintSocket.DrawLabelQrCode(element.json);
28                 break;
29             case "barCode": // 条形码打印
30                 res = await this.nMPrintSocket.DrawLabelBarCode(element.json);
31                 break;
32             case "line": // 线条绘制
33                 res = await this.nMPrintSocket.DrawLabelLine(element.json);
34                 break;
35             case "graph": // 图形绘制

```

```

36         res = await this.nMPrintSocket.DrawLabelGraph(element.json);
37         break;
38     case "image": // 图像打印
39         res = await this.nMPrintSocket.DrawLabelImage(element.json);
40         break;
41     default:
42         console.error("Unsupported element type:", element.type);
43         return false;
44     }
45     if (parseInt(JSON.parse(res.resultAck.errorCode)) !== 0) {
46         console.error(`Failed to draw ${element.type}:`, res);
47         return false;
48     }
49 }
50 return true;
51 }

```

3.4.2 二维码绘制 DrawLabelQrCode

代码块

```

1  export default class NMPrintSocket {
2      /**
3       * 绘制带logo的二维码。
4       * @param {*} json - 包含二维码信息的JSON对象。格式如下:
5       * {
6       *     "x": number, // x轴坐标, 单位mm
7       *     "y": number, // y轴坐标, 单位mm
8       *     "height": number, // 二维码高度, 默认宽高一致
9       *     "width": number, // 二维码宽度, 单位mm
10      *     "value": string, // 二维码内容
11      *     "codeType": number, // 条码类型:
12      *         // 31: QR_CODE
13      *         // 32: PDF417
14      *         // 33: DATA_MATRIX
15      *         // 34: AZTEC
16      *     "rotate": number, // 旋转角度, 仅支持0、90、180、270
17      *     "correctLevel": 2, // 纠错级别, 取值范围1-4, 默认2
18      *     "logoBase64": ": string, // logo的base64编码(不含数据头, 如
19      *         data:image/png;base64,)
20      *     "logoPosition": ": 0, // logo的位置, 取值范围0-4, 默认0:居中, 3右下·
21      *     "logoHeight": number, // logo高度, 单位mm, 默认10mm
22      *     "logoScale": 0.25, // logo缩放比例, 取值范围0-0.33, 默认0.25
23      * }
24      */
25      public DrawLabelQrCodeWithLogo(json: String): Promise<any>

```

3.5 线条绘制 DrawLabelLine

代码块

```

1  export default class NMPrintSocket {
2      /**
3       * 绘制线条。
4       *
5       * @param {Object} json - 包含线条信息的JSON对象。格式如下:
6       * {
7       *   "x": number, // x轴坐标, 单位mm
8       *   "y": number, // y轴坐标, 单位mm
9       *   "height": number, // 线高, 单位mm
10      *   "width": number, // 线宽, 单位mm
11      *   "lineType": number, // 线条类型: 1:实线 2:虚线类型,虚实比例1:1
12      *   "rotate": number, // 旋转角度, 仅支持0、90、180、270
13      *   "dashwidth": number // 线条为虚线宽度, 【实线段长度, 空线段长度】
14      * }
15      *
16      * @return {Promise} 返回一个 Promise, 解析为绘制线条的结果
17      *
18      * @description
19      * 1. 绘制元素前, 必须先初始化画板
20      */
21      public DrawLabelLine(json: String): Promise<any>
22  }

```

代码块

```

1  //返回数据示例
2  {
3      "apiName": "DrawLableLine",
4      "resultAck": {
5          "errorCode": 0,
6          "info": "draw line success!",
7          "result": 0
8      }
9  }
10
11 /**
12  * 打印元素处理方法
13  * 支持多种打印元素类型, 按顺序执行绘制操作
14  * @param elements - 打印元素数组
15  * @returns 是否全部元素处理成功

```

```

16  */
17  private async processPrintElements(elements: any[]): Promise<boolean> {
18      if (!this.nMPrintSocket) return false;
19      console.log("elements", elements);
20      for (const element of elements) {
21          let res;
22          switch (element.type) {
23              case "text": // 文本打印
24                  res = await this.nMPrintSocket.DrawLabelText(element.json);
25                  break;
26              case "qrCode": // 二维码打印
27                  res = await this.nMPrintSocket.DrawLabelQrCode(element.json);
28                  break;
29              case "barCode": // 条形码打印
30                  res = await this.nMPrintSocket.DrawLabelBarCode(element.json);
31                  break;
32              case "line": // 线条绘制
33                  res = await this.nMPrintSocket.DrawLabelLine(element.json);
34                  break;
35              case "graph": // 图形绘制
36                  res = await this.nMPrintSocket.DrawLabelGraph(element.json);
37                  break;
38              case "image": // 图像打印
39                  res = await this.nMPrintSocket.DrawLabelImage(element.json);
40                  break;
41              default:
42                  console.error("Unsupported element type:", element.type);
43                  return false;
44          }
45          if (parseInt(JSON.parse(res.resultAck.errorCode)) !== 0) {
46              console.error(`Failed to draw ${element.type}:`, res);
47              return false;
48          }
49      }
50      return true;
51  }

```

3.6 绘制图形 DrawLabelGraph

代码块

```

1  export default class NMPrintSocket {
2      /**
3       * 绘制图形。
4       *
5       * @param {Object} json - 包含绘制图形信息的JSON对象。格式如下：

```

```

6      * {
7      *   "x": number, // x轴坐标, 单位mm
8      *   "y": number, // y轴坐标, 单位mm
9      *   "height": number, // 图形高度, 单位mm
10     *   "width": number, // 图形宽度, 单位mm
11     *   "rotate": number, // 旋转角度, 仅支持0、90、180、270
12     *   "cornerRadius": number, // 圆角半径, 单位mm, 暂不生效
13     *   "lineWidth": number, // 线宽, 单位mm
14     *   "lineType": number, // 线条类型: 1:实线 2:虚线类型,虚实比例1:1
15     *   "graphType": number, // 图形类型: 1:圆, 2:椭圆, 3:矩形 4:圆角矩形
16     *   "dashwidth": number // 线条为虚线宽度, 【实线段长度, 空线段长度】
17     * }
18     *
19     * @return {Promise} 返回一个 Promise, 解析为绘制图形的结果
20     *
21     * @description
22     * 1. 绘制元素前, 必须先初始化画板
23     */
24     public DrawLabelGraph(json: String): Promise<any>
25 }

```

代码块

```

1  //返回数据示例
2  {
3      "apiName": "DrawLableGraph",
4      "resultAck": {
5          "errorCode": 0,
6          "info": "draw graph success!",
7          "result": 0
8      }
9  }
10
11 /**
12  * 打印元素处理方法
13  * 支持多种打印元素类型, 按顺序执行绘制操作
14  * @param elements - 打印元素数组
15  * @returns 是否全部元素处理成功
16  */
17 private async processPrintElements(elements: any[]): Promise<boolean> {
18     if (!this.nMPrintSocket) return false;
19     console.log("elements", elements);
20     for (const element of elements) {
21         let res;
22         switch (element.type) {
23             case "text": // 文本打印

```

```

24         res = await this.nMPrintSocket.DrawLabelText(element.json);
25         break;
26     case "qrCode": // 二维码打印
27         res = await this.nMPrintSocket.DrawLabelQrCode(element.json);
28         break;
29     case "barCode": // 条形码打印
30         res = await this.nMPrintSocket.DrawLabelBarCode(element.json);
31         break;
32     case "line": // 线条绘制
33         res = await this.nMPrintSocket.DrawLabelLine(element.json);
34         break;
35     case "graph": // 图形绘制
36         res = await this.nMPrintSocket.DrawLabelGraph(element.json);
37         break;
38     case "image": // 图像打印
39         res = await this.nMPrintSocket.DrawLabelImage(element.json);
40         break;
41     default:
42         console.error("Unsupported element type:", element.type);
43         return false;
44     }
45     if (parseInt(JSON.parse(res.resultAck.errorCode)) !== 0) {
46         console.error(`Failed to draw ${element.type}:`, res);
47         return false;
48     }
49 }
50 return true;
51 }

```

3.7 绘制图像 DrawLabelImage

代码块

```

1  export default class NMPrintSocket {
2      /**
3       * 绘制图片。
4       *
5       * @param {Object} json - 包含绘制图片信息的JSON对象。格式如下:
6       * {
7       *   "x": number, // x轴坐标, 单位mm
8       *   "y": number, // y轴坐标, 单位mm
9       *   "height": number, // 图片高度, 单位mm
10      *   "width": number, // 图片宽度, 单位mm
11      *   "rotate": number, // 旋转角度, 仅支持0、90、180、270
12      *   "imageProcessingType": number, // 图像处理算法, 默认0
13      *   "imageProcessingValue": number, // 算法参数, 默认127

```



```

14      *   "imageData": number, // 图片base64数据, 不含数据头
15      *           // 如原始数据
      为"data:image/png;base64,iVBORw0KGgoAAAANSU"
16      *           // 传入的数据需要去除头部, 数据为, "iVBORw0KGgoAAAANSU"
17      * }
18      * @return {Promise} 返回一个 Promise, 解析为绘制图片的结果
19      *
20      * @description
21      * 增加接口说明:
22      * 1. 绘制元素前, 必须先初始化画板
23      */
24      public DrawLabelImage(json: String): Promise<any>
25  }

```

代码块

```

1  //返回数据示例
2  {
3      "apiName": "DrawLableImage",
4      "resultAck": {
5          "errorCode": 0,
6          "info": "draw image success!",
7          "result": 0
8      }
9  }
10
11 /**
12  * 打印元素处理方法
13  * 支持多种打印元素类型, 按顺序执行绘制操作
14  * @param elements - 打印元素数组
15  * @returns 是否全部元素处理成功
16  */
17 private async processPrintElements(elements: any[]): Promise<boolean> {
18     if (!this.nMPrintSocket) return false;
19     console.log("elements", elements);
20     for (const element of elements) {
21         let res;
22         switch (element.type) {
23             case "text": // 文本打印
24                 res = await this.nMPrintSocket.DrawLabelText(element.json);
25                 break;
26             case "qrCode": // 二维码打印
27                 res = await this.nMPrintSocket.DrawLabelQrCode(element.json);
28                 break;
29             case "barCode": // 条形码打印
30                 res = await this.nMPrintSocket.DrawLabelBarCode(element.json);

```

```

31         break;
32     case "line": // 线条绘制
33         res = await this.nMPrintSocket.DrawLabelLine(element.json);
34         break;
35     case "graph": // 图形绘制
36         res = await this.nMPrintSocket.DrawLabelGraph(element.json);
37         break;
38     case "image": // 图像打印
39         res = await this.nMPrintSocket.DrawLabelImage(element.json);
40         break;
41     default:
42         console.error("Unsupported element type:", element.type);
43         return false;
44     }
45     if (parseInt(JSON.parse(res.resultAck.errorCode)) !== 0) {
46         console.error(`Failed to draw ${element.type}:`, res);
47         return false;
48     }
49 }
50 return true;
51 }

```

3.8 标签预览 generateImagePreviewImage

代码块

```

1  export default class NMPrintSocket {
2      /**
3       * 生成图像预览图像。
4       *
5       * @param {number} displayScale - 图像显示比例，表示 1mm 的点数，可调整预览图大
        小。
6       *
7       *                                     例如，200dpi 的打印机可设置为 8，300dpi 的打印
        机可设置为 11.81。
8       *
9       * @return {Promise} 返回一个 Promise，解析为生成图像预览图像的结果
10      *
11      * @description
12      * 1. 在调用此函数之前，必须确保图像数据已准备好，否则无法生成预览。
13      */
14      public generateImagePreviewImage(displayScale: Number): Promise<any>
15  }

```

代码块

```
1 //返回数据示例
2 {
3     "apiName": "generateImagePreviewImage",
4     "resultAck": {
5         "errorCode": 0,
6         "info": "{\n\t\"ImageData\" :
\n\t\"iVBORw0KGgoAAAANSUhEUgAAAAZAAAADwCAIAAAChXqV1AAAgAELEQVR4AezBeaznd33f++fr8/2db
Wb0eMYzY+MZb+DxgAk2lN5QGivRC71SkyqtKhVVAZEAOUsQ6U1RK4SaRKJqdRM1EamKyI3C4pKNViit
q1QJ/aMNNA2JaEoANxhsg5fx2B4vs585y+/7eV7zTY80R/POVUa3FhrpPB5R2bVr167rQVR27dq163o
QlV27du26HkRl165du64HUdm1a9eu60FUdu3atet6EJVdu3btuh5EZdeuXbuuB1HZtWvXrutBVHbt2r
XrehCVXbt27boeRGXXr127rgdR2bVr167rQVR27dq163oQlV27du26HkRl165du64HUbKWSaioVJJQU
akkoaJSSUJFpZKEikoLCRWShIqKpUkVFQqSaioVJJQUakkoaJSSUJFpZKEikoLCRWShIqKpUkVFQq
SaioVJJQUakkoaJSSUJFpZKEikoLCRWShIqKpUkVFQqUbKWSaioVJJQUakkoaJSSUJFpZKEikoLCRW
VShIqKpUkVFQqSaioVJJQUakkoaJSSUJFpZKEikoLCRWShIqKpUkVFQqSaioVJJQUakkoaJSSUJFpZ
KEikoLCRWShIqKpUkVFQqUbKWSaioVJJQUakkoaJSSUJFpZKEikoLCRWShIqKpUkVFQqSaioVJJQU
akkoaJSSUJFpZKEikoLCRWShIqKpUkVFQqSaioVJJQUakkoaJSSUJFpZKEikoLCRWShIqKpUkVFQq
UbKWSaioVJJQUakkoaJSSUJFpZKEikoLCRWShIqKpUkVFQqSaioVJJQUakkoaJSSUJFpZKEikoLCRW
VShIqKpUkVFQqUbKWSaioVJJQUakkoaJSSUJFpZKEikoLCRWShIqKpUkVFQqSaioVJJQUakkoaJSSU
JFpZKEikoLCRWShIqKpUkVFQqSaioVJJQUakkoaJSSUJFpZKEikoLCRWShIqKpUkVFQqSaioVJJQU
akkoaJSSUJFpZKEikoLCRWShIqKpUkVFQqUbKWSaioVJJQUakkoaJSSUJFpZKEikoLCRWShIqKpUk
VFQqSaioVJJQUakkoaJSSUJFpZKEikoLCRWShIqKpUkVFQqSaioVJJQUakkoaJSSUJFpZKEikoLCRW
VShIqKpUkVFQqUbKWSaioVJJQUakkoaJSSUJFpZKEikoLCRWShIqKpUkVFQqSaioVJJQUakkoaJSSU
JFpZKEikoLCRWShIqKpUkVFQqSaioVJJQUakkoaJSSUJFpZKEikoLCRWShIqKpUkVFQqUbKWSaioV
JJQUakkoaJSSUJFpZKEikoLCRWShIqKpUkVFQqSaioVJJQUakkoaJSSUJFpZKEikoLCRWShIqKpUk
VFQqSaioVJJQUakkoaJSSUJFpZKEikoLCRWShIqKpUkVFQqUbKWSaioVJJQUakkoaJSSUJFpZKEiko
LCRWShIqKpUkVFQqSaioVJJQUakkoaJSSUJFpZKEikoLCRWShIqKpUkVFQqSaioVJJQUakkoaJSSU
JFpZKEikoLCRWShIqKpUkVFQqUbKWSaioVJJQUakkoaJSSUJFpZKEikoLCRWShIqKpUkVFQqSahcv
rwxn2/05/0trS1Aba0BR44cofLCC88lCzCXEVsyQDftxgMHqZw9e5ZJEjUT9YYbbqBy9uzZJCqQBEgC
7N+/n8rZs2eBJCqQhMkNN9xA5dy5c733JGomTg4ePEjl7JnTI7MkdIHIONNx8ciNq1See+45rqAC6k0
33URFpZKEikoLCRWShIqKpUkVFQqSaioVJJQUakkoaJSicq1SEJFpZKEikoLCRWShIqKpUkVFQqSa
ioVJJQUakkoaKy0wgb5y/svWE/lbNnz6pMMmFyww03ULl8+TLQJkDvfRgGYDabUVHZpgJJ1NYaFRVQg
SRM1NYaFZVKEioqoAJJABVorVGZz+dJeu/qMAx711dXFykcvHiRcBtgAoc0HCAygsVvKDyoiwwdtPN
XL3p8M1UVCpJqKhUklBRqSSho1JJQkWlkoSKSiUJFZVKVK5FEioqlSRUVCpJqKhUklBRqSSho1JJQkW
lkoSKSiUJFXXtt3/n3GvvWZ4Nw8rSrA2ttWEYFhcXqaiACiTpvbFWHnvssTvvvJ0KyjY1CZMkV0bzOd
B7d9J7d7Jv3z4qzz33nBPAbb33Y8eOUXn44YeB3ruTP1HvvfdeKp/730d67/P5XJ3P530yn8//5t/8m
1Tuv//+PmGnd7/73VTuv//+J0revXs3Njb0nj27urp65MiR7//+76cyn8+dJFHn83lrTV1ZWaFy8uRJ
4NixY+yUhIpKJQkVlUoSKiqVJFRUKkmoqFSSUFGpROVaJKGiUklCRaWShIpKJQkVlUoSKiqVJFRUKm
oqFSSULl48WISFWitDcOQycLCApXe++bmprq0tJTkYx/7RJLz58//w3/4f1E5fPjw8vLyysrKMAyttD
47kwcfJDKK17xKnUcR7eN49h7P336NJXl5WV3SgLM53MqS0tLTDJprQGttQsXLlC56aabgDYBwmtJg
CeeeILKPFfcA7TWkrTWkrTWkvYp//E/qHz3d393ay3J/v37V1dXDx06tLq62lr7Z//sn1E5d+5c7z3J
k08+edddy0uLiZRW2tUVCbqOI7qwsICkISKSiUJFZVKEioqlSRUVCpJqKhUklBRqUTlWiSho1JJQkW
lkoSKSiUJFZVKEioqlSRUVCpJqKhUklBZW1tLwrZhGJIACwsLV066666tra09e/ZcuHDh4sWLW0Li4s
LCwqlTp6jccsst6sbGxmw+T6ImAc6fP0/l80HDSVprSVprSYAKJ0+epHLnnXcmAdQkgAo8+uijVG655
Rag9+50L7zwApXV1VU1CZAESAkc03e0yk033dRaA1prucLjjz905fjx40CSzYmaBHjqqaetzGazD37w
g+9973s/85nPFoELX3jta1+7Z8+eo0ePvvGNb6SiUklCRaWShIpKJQkVlUoSKiqVJFRUKkmoqFSici2
SUFgpJKGiUklCRaWShIpKJQkVlUoSKiqVJFRUKkmorK2tqcMwJPnYxz721/7aDxw7dkubULnvvvuA3v
uLS5fW19dXV1cXfXfPnj178uRJKi972cvW19fHcWytAa01JmfOnKFy6NAHJioTJ2fPnqWyD+9eoPfOt
```

iTA2toaLUOHDjFprWUCJHn66aep3HbbbUycAE6eeuopKocPH3bCNhU4c+YmLcOHDzNRmahJnn/+eSor
KytqEiAJkMnFixepqFSSUFGpJKGiUklCRaWShIpKJQkVLuOSkiqVqFyLJFRUKkmoqFSSUFGpJKGiUkl
CRaWShIpKJQkVLuOSkr3397///T/5kz+5urr62c9+9i/+xb/4oz/6oydPnvxP/+k/Ufme7/me3vvGxs
a5c+c2Njb27Nkzm81673/8x39MZxV1dXFxcRgGtqnAs88+S2X//v1uA1Qma2trVFZXV4EkQBK2nTt3j
srq6iqgsi2JevHiRSr79u1jkoQrXLhwgcr+/fu5QhI1yblz56gcOHAASOIEUIHz589T2bdvH5WLFy9S
+ehHP/pbv/VbP//zP3/rrbc6aa0BSaioVJJQUakkoaJSSUJFpZKEikolCRWVSLR2ffvM5/Nnn30WWF5
eHobhN3/zN7/85S//9E//NPBHf/RHt99++1/6S3/p0KFD8/l8bW3t3LlzeybDMLBTEvXy5cvnzp1bXL
4+cODAMaXMKD3vrm5ef78+eeff35tbU2dzWaAylVUrKUSkip/OjUJExViojJJoiZhogJJADUJ0yXpv
QOZqExUtrXWVP4UKtuSACqTJGoSIAnghEkSKkm4SpLe+1/4C3/hM5/5zEc/+tHXve513/zmN7/whS98
6EMfYqImOXpmzMGDB9m1U1R2ffucevIJMsxms49//OOPP/74Rz7yka985Sv33nvvF77whbe//e2z2cz
J1tbWpUuXxnFcWkBXnFUx3Hsvavj0Krz+Xwcx957EhVQgd67ylWScIUkKjslUYEkfFOBJPYzqUmAJC
oTNQmgAkkaLUkSlaskUZMwUJZPwZ5CEbWoStqmtNa6gJvEKgNpaG4YhiQqoSZgk4SpJuEISrqCurq4+9
NBDn/rUp17/+v/j137tV37u5360yebm5rlz544c0cKuK0RL17fJxceefGEpfX3z4x//+Hvf+9719fXb
b7/92LFjFy9e3LNnzxve8IZ77713ZWVlc3Pz6aefPnny5NGjRw8fPgyovXegT+bzee/9/Pnzjz32mHr
s2LG9e/eq8/m89z50LL68+PDDdz/55JPr6+tAktaayv8mKpCE/9+SMFGTqEn4M30SR0UKklDQk6hAEp
U/gyTDMLTWAJUrJOEKsDgpCTsLUVtrN91006c//RsPP/zw3Xff/bWvffVv/+2/3Xs/c+bMoU0H2HWFq
Oz6Nnn69DMf+Cf/5BW33/GTP/mTm5ub//Sf/tOf+ZmfAXrvs9ksyd69e0+cOPGGN7xhfX19YWHh7rvv
XllZUcdxVMeJurW1tbm5eerUqYceeuJqoU033nprknEyn883J88999wDDzzwzDPPz0dzJklms1lrLQm
QBEgCJGGShCskYVvvPQmg8qdQkzBRgSRMVLylUZMwUZOoSdimJlGTsE3lKiQGAr13tjkBnAAq21T+bI
ZhaK0BKldIwiQJ0yVhpyRM1EyAX/zFX/qDP/iDf/SP3veLL33pTW960/r6+qFDh9h1hajs+jZ57z/4s
ZuP3PQTP/ETJ0+e/OAHP3j//fcn4QqLi4v33Xffn//zf/7y5ctHjx59+ctf3lpzMo5j730cx967ur6+
/sgjjzzxxB033377LbfcMgxD730cxz7Z2tp66qmnPve5z506dWocRxVora2uri4sLayT2WzWJsAwDG2
ShEkSrQACKh0Vidp7T7K5ufmNb3yj9/7hD3/4X/7LfwN03t/+9rf/1E/91Gw20378+Pr6+g/90A998I
Mf7L2rvfckvXeVyU033QR8+MMf/lt/62+N46g++OCD3/M93z0bzb2ts6fP7+1tfWbv/mbP/IjP9J7T
6JyFTUJoCZhJ5WrqOyUhg2tNaC1xk5JuEISKkm4QhImSZh853d+55vf/H++5S1vOX366XvvvffAgQP
sukJUdn2b/I2/8Tc+9KEP9d6PHz++sLCgzmazYRj6ZBzHG2+88S1vecuJEyFW19df8YpXHD58uLWmbk2
SAPPJ2traV7/61V0nTt1zzz033nhj732+bWtra2Nj47HHHvut3/qt06dPj+PIZBiG1dXVYRjaTklaa9
kGtNaYJAGSqICapPeuMlGTjOMIOPmxH/uxH//xH++T97///Z/61KeAcRyTPP7448Dtt99+6dKlhYWF+
Xzut67mutf/Jt/873f+73PPPPMa17zmmeffbb3nuTo0aNPpfVU7/3jH//4Bz7wAZVtSXgptdYy4SpJ
2JaEnZJwLSRsywR417vedeedr3jPe97zu7/7X9/85jex6wpR2fVt8ta3vvXXfvXXzl84//f//t//t//
23/beh2EAnABJhmFQgTYBVEBlogJ0gGEYXvayl33Xd33X3XffvbCw0Ht370vr6w9+/Wu/8zu/c/78eZ
XJ0tLSn/tzF251dXV5eXlpawllZWVxcXFhYaFnlpexW2uz2ay1NgxDmyRprSUBMGsSqExUJr13t12+f
Ln3nqRPtra25pONjY3FxcWFhQUmSba2tlprw1tbrbV//+//fZK//Jf/8uHDhy9duvTrv/7r4ziqb5os
LS0Nw/Af/+N//MxnPu0ktQaoSXgJJGHSWkvCFZJQSQIkoZKESRImSd19fW/83f+3r/4Fz/TwtuzZw+
7rhCVXd8m7/nRv/eRj/zixUsX/+pf/au//u/r3KNVP50SdimcgU1CX9mSVT+PyWh8h/+w39YXV2955
57TpW48cADD3z4wx9+3/ve9+lPf/rnf/7nH374YfWFF1648cYbl5aW1tbWkgCbm5uXL19+y1ve8slPf
vJlL3vZuXPnNjc3jxw5Apw+ffqmm24CXvOa1/zxH/8xf4ok/G+ShG25AldJwrYkXCUJkEQFkjBJwhV+
6Zd+6U1vetPBgzcuLMzYdYWoXIskVfQqSaioVJJQUakkoaJSSUJFpZKEikolCRWVq7ztB9/6a5/6dSr
DMKhsU5moVJJQUakkuBlGSdhJpZKESZLjx49/9rOfnc/n3/Ed33H+/Hkqv/qrv/rII4/83b/7d48ePX
r58uWf+7mfe8973nPw4EGVvyLy8ubmpspOKpXWGPxe05XWGPmkXGECrypLS0tJgCRsS7K2tkZl7969Q
BIgCZMk58+f56Wkcj2LyrVIQkwlkoSKSiUJFZVKEioqlSRUVCpJqKhUklBRucp3fdcbP//5P6Aym81U
rjK0I5XWmspVVCpJ1CTspFJJwhXUJIBKJQmQpPd+80DBT37yk//u3/273/7t3z516hSVJIAK/Jf/8l/
uvPP0j3zkIz/7sz+rUvnEJz7xrne9i6uoVFprVHrvVFprbEsCJAhm8zmVLZUVJkmYJFHx1taorK6uMk
nCfc6fP89LSeV6FpVrkYSKSiuJFZVKEioqlSRUVCpJqKhUklBRqSShonKVRx555Pjx41QWFxd770xUJ
mrvnUoSKiqVJFRUKkmoqFSSUFGpTNaO9N6pDMNAZRxHkrPZDFCTAGoSYD6fU5nNZlwlydbWfPwVLZuk
QBImSYCLFy9S2b9/P5MkQBIgyZkzZ3gpqVzPonItklBRqSShoLJJQkwlkoSKSiUJFZVKEioqlSRUVK7
y1a9+9dWvfjWVpaUlwG1sm8/nVFprKldRqSShoLJJQkwlkoSKSiUJFZVKA41K753KMAxUxnGkMpvN2J

YEyGRjY4PKvn37mCRhkgQ4f/48lQMHDiRhogKbm5snT5688cYbeSmpXM+ici2SUFgpJKGiUklCRaWSh
IpKJQkVlUoSkiqVJFRUDprP5w899NCrX/1qKsMwLCwstNbU3rsTYGtri0prjSuoTFQqSaioVFprVHrv
VFprVHrvVFprVHrvVFprVHrvVIZhYFsSts3ncyolCwtMkjBJAmxsbFBZXV1lkgmQyZkzZ6gcOnQIyLZ
Pf0IT3//933/+ /PkbbriBl5LK9Swq1yIJFZVKEioqlSRUVCpJqKhUklBRqSShoLJJQkVlp7W1tccff/
yee+6h8vu///vnz5//vu/7vuXLZWAYhiSttTNnzLDZv3//OI7z+RxwW59QyYSr9N6ptNao9N6pDMNAZ
RxHKsMwUBnHkcpsNqMyn8+pLCwssFMSYHNzk8ri4iKTTIBM1tbWqBw4cABIAMTC5Pnnn6dy880303n6
6adbaxsbG48//vgf/uEf/uAP/iAvJZXrWVSuRRIqKpUkVFQqSaioVJJQUakkoaJSSUJFpZKEisp0Fy5
cePrpp0+c0EHlda973ec///nl5eX/9t/+25kzZ376p3/661//epLTp09TOXDgQ099Pp8DSZiM47i+vk
6ltUaL905lGAaukmQ+n10ZzWZU5vM5lYWFBSpbW1tUFhcXqWxubLJZWlpiWxK2ra+vU1lZWQFaa2oma
mvtwoULVA4dOsSktZZEzeSZ56hogKnTp3at2/ft/zET7z1rW9dXFwchuF1r3sdLyWV61lUrKUSKiQV
JFRUKkmoqFSSUFGpJKGiUklCRaWShIrKTmcmd911F5UHHnggSe/9fe9734c//OETJ0586EMfetvb3nb
zzTdTueGGG9RxHNUkbltfX6cyDANXSMJkPp9Tmc1mSbjK1tYWlYWFBBYlYdvm5iaVxcVFdkoCbGxsUF
leXqayvr50Zc+ePeyUBLh06RKVffv2AUmYZNvZs2ep3HzzzUySAK01JqdOnaLysz/7s1/60pf+wT/48
fl8M0lrbTabtdZe97rX8VJSuZ5F5VokoaJSSUJFpZKEikolCRWVShIqKpUkVFQqSaio7HT690nLly/f
eedVB566CFgHMckvffWwu99Pp/fe++9VG688cbW2ji0vXci1ZMLFy5Qmc1mQBJ22traorKwsJAEULn
C1tYwLcXFRSqbm5tUlpaWqGxsBFBZWVnhKknW1tao7Nu3TwwSAEmYXLhwgcr+/fuZZAJk8sILL1A5ev
RoJkBrLcmRI0f+83/+z/v376fy3//7fx/HMVdpk9lstrC0eNfLX8GunaJyLZJQUakkoaJSSUJFpZKEi
kolCRWVShIqKpUkVFR20nXq1Hw+v+0006g88cQTKtB7d5JkHMFjx49TUR977LEPfoADv/ALv6DecMMN
wIKtJx5++GEqS0tLTpJwhc3NTSrLy8uAyrYkwPr60pWlpSW1tQaobNvY2KCytLTEJAlXWF9fp7KyspK
EKyQBLl26RGVldZWdkgDnz5+ncvDgQSYq0FrL5LnnnqPyR3/0R6997WvVz3/+869+9asffvjh97//A/
P55uc+9zkqf/iHf8i21lomrbUkwzAACwsLd911F7t2isq1SEJFpZKEikolCRWVShIqKpUkVFQqSaioV
JJQUdnpiccfF+644w4qTz31lCaRiZpEPXr0KJVTp04NwwDM5/NbbrkF+0t//a+//OUv/1f/6l9R+Yef
+IHW2jAMKpPWWpJPf/rTVN72trcBrTWgbUvy0Y9+lMpP/dRPzWazhckwDLPZbGFhYTabvfvd76bye7/
3e6212WY2sLAWmywsLAZDcMcdd1CZz+cqExXovasrKytUvvGNb4wTtU9U4L777qPy4IMPttaGYdja2h
only9fvnDhwL/5K3+Fyjev+c5//I//8cte9rKTJ0/OZjNga2sryX333Ufli1/8oportNaStNaGYQCSH
D9+nF07ReVaJKGiUklCRaWShIpKJQkVlUoSkiqVJFRUKkmoq0z0jW98Y2Fx+fbbjLE5c+YMNHUYht57
a20Yoq6urLJ5/vnnx3EExnHsvQNqa+3o0aUnnvuuXEck6iZqEmOHDLC5fTp05moXOHikSNunvu0SY
qoCYBjhw5QuWZZ55JAiRRgdaaeuTIESqnT59moiZhkuTIkSNUnn32WZUrJJNp57fccguVp59+urUGj0
PoB0i933bbbVS+9rWvJVEBJ0mAV73qVVS+8pWvMFGTtNaybRgGQL377rvZtVNURkUSKiQVJFRUKkmoq
FSSUFGpJKGiUklCRaWShIrKTg899NDKysptt91G5eLapSFNTTIMg8pkaWmJytrausrEbUlWV/dSef75
55kkUZ0owKFDh6g899xzSQAnQBLg80HDVE6fPp1ETQKoQJIjR45QefbZZ1UmKpMkn910E5Wnn35aBZK
wTb3l1lloPPPMM2rvPQmgAkluueUWKqdOnVKZJ0m9M7n11lupPP7440nUJICTJLfffjuVr3/960nGcQ
RUQE0yDENr5EXw8lccZ9dOubkWSaioVJJQUakkoaJSSUJFpZKEikolCRWVShIqKjt985uPLC0uHrv1d
irjx1prDbAFUJMAbvii0t2MqICaDE5mC0tU5lsbahIVaKICs+UVKhvra6213ruaBFBba4tLK1TW19eZ
qEASJ3v27KGytramMLGBJMDvXupXLp0SWXSwgNUY0/evVQuXboEDMPQewd676213vu+ffuoXLhwQeV
bGqAmAfbv30flZJkzKt/SAJXJoUMHqTz66KPQ/4QTIJCKtZYEUPPlD7Frp6hciyRUVCpJqKhUklBRqS
ShoLJJQkWlkoSKSiUJFZWDHv3mI4uLi8duvZ3KfP3SMaYALS/iRQmQLFDRkUoyU0l9nkrLEv6XtBmVP
m4kAdQkKpM2LFHp4wYTNyMaP9EWqYzzdRVora1tGPo4qrOFFSrzcrcSDAdQkgC1AG5ao9HEDUJk06eFF
w2yZyrixsiSJkyTqbHkvlfXLFwx0uULvfe++A1Se0nWy7+SktZZETfLyVxxn105RuRZJqKhUklBRqSS
hoLJJQkWlkoSKSiUJFZVKEioq0z36zUcWFxeP3Xo7lfn6pWEYSaXJ+BN50QIVFToTNQmTZKCiI47j6D
AMJNDtXR1my1Tsm4CaRGXsmQ1A2iIV++Y4jq21+XzeWuGofcODLNlKv0ty621tGbvahKg9z5bWKEy3
7qcbpIeBtJ7twWYLaxQcWtdZZJEBdRhaQ+VvnLZBZIAKpNhaQ+VjFWLTdR5pAuk0+K+1YNUTj35hNon
6ji0vXcgsWsNaK3dcecr2LVTVK5FEioqlSRUVCpJqKhUklBRqSShoLJJQkWlkoSKyK6Pfv0RxcXFY7f
eTmW+fsmWWRoJLbwoAZIFKn7LmERNBuiA2tqMim7htyQhsfckahuWqPRxo/fewLn778MwRDOs2Uq43
y9tab23ofZrI9jEnWYLV0Zb10G1GEYeu9JAHW2sEKljxtM1CRQxtTNwjKVvnkZSNJ7b8Ng70za4gqVc
WNNba2pQ0+doTUZlvZQ2Vy7AKjAiC9Kou7dd4DKU6d0qr33cRx77+o4jsDdJ17FS0nlehaVa5GEikol
CRWVShIqKpUkVFQqSaioVJJQUakkoaKy08lHH2uNY3fcSWW+finbbAGSkBctUNFRBZJwhWSgMo5bLfb

QjnMH0vaBIzmVUVHNSph5ozr68VYGMdxRVljCcVwcCFzPL6ujrGUQrXWwkpRh6NLmTPeuJoEqMRJMRu
rC5czZ7xxVR1KqTpGNqobBxfvyZz19StJABVioiZZXbicOeONqyqgMqm1AsPRpcxZX78CqJmoSYDVhc
uZc+3qy5k4SaLUWt903wOZ8/xzzyQZx7E2ahKglKLe/8D9Dz74cLrXQ802gMxRMwfIHDVzgMxRMwfIH
DVzgMxRMwfIHDVzgMxRc4uvPv3lg4NhWB0++ujbMufmjStA2TCVMElC0cwc680aCyuV1EC1Yspwldl1
vKEmqbUySQKU4Shz6s1rSSwU40aBapJyeDFzxhtXcwsVSDIcXcq8cbVJGoppdYKZDIcXcqcmzeuDGH
DCeDk40I9mb0+fiUJoGZSSLGHo0uZc3ztFSCJCLTys0jy5lz7erLSWwyqTVvuu+BzPna88+q6/Varb
U6qbWWSa31zQ8990YHHkz3eqjZBpA5auYAmamDpA5auYAmamDpA5auYAmamDpA5am7x3NNfYZVy4
ehtD/9Q5ow3rpZSKgGSAJlQDjPHutaRUqKZ1FpLKZTDzBnX14EkGGSJCajl8GLmeHw9G3A8roEkQ3jV
wYXMWV+/UkqptWYC1FqHYSiHFzNnvHG1ErUYNpJaa1bD6uBi5ty4/gqQZgi1VmB14XLmjDeullK06wi
k4FiHsFEOL2b0jeuvZDKESkq1FopZXbic0deuvoxEkuprktRa33Tfg5nzteefrbdIUmvNBEjy8EMP33
v//eLeDzXdD9xXn/7ywcGwWh2uDodSykAZNgKrYQhDKUkoJZBEAiSREJKSV9VsaBpJqmmATGqtmVctZ
EMtG+bPDSVjVTMUCTAer7/zne889dS/e0bZ5+PxQw899M4nHn/LW95yMKyGUMexDEMSYG1Vh1BKWa/X
maxWq3EcgSQqkKTWmqFQLVAVSKMCKpBEBVQgiQqoJCaVqAdlGMexlKIC0WPrASXJSIAKQ1BJqg7DEEH
hHMchBNZ1XFFqRumASlSqRR1jEmollKSjDEJ1QJVK9mgmskYM5GkulFrTVJrLWW1Xq+d1FqTj00oju
PopDZqkn/5xx/PktTsMtR0P3BfffrLh4erYTgYDspAWa1WQyilrIbBpJTCa0pJItkAMPeQjEnIoIaah
GCTgAKmGceRiVprTcL3GMklf/bif/vf/Nrf//v/1fHNmxnXpZQxAK6GkGSMsVaF0QzDwZ889/wn/49P
/vW/+teAJKwGN9ZjmYzjmaQ4ruOKopKYALVWoJRSawWSAGNMUgzJ0qbgWfCufVArGUISQK21AtFaeFV
1K0UYhxoSSqnkk5/85JNP/p8f/shH1sfH0masbMQUNNREMwxDKauvfOUrv/Vbv/Whv/HvP/zw2NMqh
XIxEk0MMZSctUxZkjljEncICWotdYktVYyxng0qRN1HEe1TtQ6SfLj73wiS1Kzy1DT/cB95ctPXbh4e
TUMZcVAGSZAAZKUUpIASYA0EoIxCSETJVRcNImaxAmvqW4A2QBr5WD1j/7Rb3zoQx8arBZqrUMYx3G1
WtVak1SiHh8fHx4eqoCaRk3yqU/93uHR0Qf+4vtrrZk4YaIctVbAjQLVSoaglllLWGEqptVJNwkrNAiS
ptQKVFENSFaillKSWFCBYqKU8k/+6Sf/pR/7sUcfeaTwdWLCMAwqoOYWgJoEUIEkWcc+8V9/5CP/2U
EZNSZYzNqaZEUYZ17PSZJaK5DESRL/HEnGcawTtTZqrTXJOI5qrTXJj7/ziSxJzS5DTfcd98UvfGey
5dLKavVahgok4MyLCZJKSUGAXIryIavSiEJRgVUQM1ELSYQX5WE1VCP10CG8l9+9GMf/9hHXY9JKimG
xER9/Cf+lS8/9YX1ej3G3/md3/ngBz+YBPi5n/u5/+0f/+NrV6+WUtiw1nDz5vrX/+E//E/+4/9oWJX
18QhYSFLMhpMCJpUmWULGWmCsNUNhXf6Ho4MbDAdr3zJc+fmsbtb1wXpIcSMNoCYBNAAwitkAysHq7/
zyf/Df/YN/U0taLeZ7Di5e+vjHP/7Nb3zj0qVLq9Xq8PDw60hgtTr88If/05s3byYBSinj0JZS10ef/
5PPff7z/+6/82/X0hbZSAKMkSowxiTFqJVsQEmptSbRMYlaazacj0PopE7GcUxSa1Vroz7+xLuzJDW7
DDXdD9wXv/iFey5fBoZhODgYymRFWQ1DYCgFqIXXZALkBEmqQBoVE82kEoljLeZ7xrharT72sV/5+K9
87PjmTbWY16i1lp/6wAd/7/c+tv6vV6tvklprklLKhZ/8n//qr/4qri011iTA1atX/8f/6X/+D3/57z
gBkqjAGIvZKKU4GeMQKhmcuSfvXL1yQcfvllzIRnH+kv1JsVSR5NK2KgClahJinkNUEKxQCubf/fv/
hef+MTfo8ppqLUCSf7J//5P3/++9/3ar/3aL/7ih5IMw5AE+JW/94mPfPjDP//zP//7n/m0CmRSyQsv
/Nnv/u7v/pW/8rMZHaMEs0E1CVCJEyATNUmt0TGJkqTwtZqUOnEyjm0tNUmtdb1e29Ra1cefeHeWpGa
Xoab7gXvmmacLHBwCAACHQyllGIYVBSilrFarJEqYZIT1CSUES2txlpJCfFVdQBD9VW10pQhZGMov/
Ebv/FLv/TLx9evWVCTFLMBqEnKweql15+8KGHjm9eX6+Ph5QkFmosYqif/8JTX/zSv/j3fvZn11Y1k
1KKCtRaATUTNRuFoQZIUzcIkAZIUsz3jDGJmkIJxVRSzEYlG8Dn/ujzv/epT//tv/2LQ0hCYnJjPf72
b//2K6+8cjwZx/H4+Hi9Xt+4cQP44Ac/+IEP/OtYUwag1vriiy/+wR/8Xz/zM3+J6qsKmahJVCATNYm
aBFBrrTDUWnVMoiRZr9fAOI7qOI5qrVWtr5cEeOfj78qS10wy1HQ/cC9tfPvFYRgODw9LyTAMPZRhGE
opwDAM6kEZMPriNlgNadQkQBIVSKJm4qSYSoozyEzEqEst5DVUk5RSMpT/5X/9ze+89NLf+lT/0/U4D
MMYqY6RoWBeozI5Pj7+6Ec/+tM//W/+zM/8pbIRSVGTVKIyUZOWU2ztpZS1DTFjCUlFLNRCeAGwaiA
CiRRgSQqoCYBktRah2FQj460fvM3f+vg40Av/+V/i2oBk0qKqerWtdYhJAFevvLKr//6f/8Lv/ALjz7
61rJhKlGTAMWoLbxGzQRQk6hJ8VWjmhS1ToD1eg3UWtfrNTB0ktRax3F0kkR94l0/kSWp2Wwo6f7/8K
UvffFgtTo40CiLDAPDBFitVqUUmLJKklJKvj+MRE2iFqMmqbVaqLxm9dQhjDEJkEQFVEcttQ7DcHx8f
OXKlVrrhQsXjo60gFrrwcFBknFcF9lQk1Ty/6ICAQa1CVBrBZK4QUpQSylqCqkmUZMAOUeF1EzYqCyp
pYwxE8lQQ3JsHYbhc5/73Le+9a3r129+97vFPT4+vnjx4oULF4aB97znPQ888ACTUopaSlFrrUlKKUm
KswADJFGBWitQa4UKJPFVJkm1JlFrrWqdqHwijuNYa1XrRE0CPPGun8iS10wy1GwDyJLUvGE8/fSXh1
KAg4Nho0xwQ1UpBVhRNsY4DAOTJICaphi1ku+hupFkjEmOLtyTM3XllZeSALXWUkqtdRiGi5fel0bKK

y+luXzP/Znzyne/DSS5fM/90VNXXnmp1lpKUZMAKpDECZBEzRxAzQRQc4shHNeRiQqoSULSa83EE2qt
6ji0ap2o6/U6iVonSdRSiro60Lh48fLb3/72dCegZhtAlqTmjeRLX/riUMpqtSqLHBwMQCnL40AgSSk
FKBOglKImATJR83pqJmqtNcmly/flTL3y3W+reb173/TmNC9/50Ugifiqm+x7MnO++/GeAeu+b3pwz9f
J3XkwC1Fp5TTVJJSqgJgHUNICaCZBEzUQF1LyqJNGRSa01iQpDklprEie1VhWotaq1VnUcxzpRa61qn
eT/USmrCxcu/OiP/li60ajZBpAlqXmD+aPP/+HBwcHh4WEpZbValZLVagWsVqtSSpJhGJgkKaV4k4gRI
oiaptQJJ1ETNxj33PpAz9Z2XXLABNQmQ5L77H0rz0re/lQnkvvvfknfeemFT067/6GcqZe+/S0134e
ahIkKqPlzJaLqGkBNsiY6JgGS1FphUDNxAi11iRqrTWTegt1HMdaq5Naa5qqLhpwePHxH38i3feBmm
0AWZKaN6TPf/4PDw6Hw9XRMAyr1QostwDKBFCsmkRJKqAmUfKqmkSpdQ0880aHc6Ze+NY3mSQ1Sa0Ve
PChR9K8+MI3kgKoDz70cOa8+MKfZvLgQw/nTL3wrW8mAZOoeVVJo2NeVdKoSYAkar4PNUkpRa21AknG
cQSSqEm8BTCOYxIn46TWmkSttapJ1FjW403K4cVL5fLFe97+wz+S7vtDzTaALEnNG9U4jp/7w392dHR
0cHCQ50DgoJQCDMOQZBgGoJSiMsLEtaICtVZArbUCSd76yGNpvvH1ryVRSym11iSlFDUJkESttQKLLC
QqoD7y6NvS/Ok3v57Jw299NH0++Y0/AdQkb33ksTTf+PrXmDgB1EcefVuaP/3m1zNRmdQJkyQqoAJJa
q2A+uhjP5TmG1//mgqogFpKcZJbqEmA3KLWCqiAmjLqEhVQkzhJoiZRa61J1Fqrk1qrWmtVa61Jaq2Z
uD7m0uEFLj7x7nelux3UbAPIktS84T355GcoXjy6BKxWqyTDMNCUUGAVUAEnQBo1k3f8hR9N88xXv5J
ETQ0oQCZAEhXILX74HT+S5rlnv5oE+KG3vyNznnv2q5moP/yOH0nz3LnfVQE1CaD+8Dt+JM2zzzydhE
mtNZNSSpJxHJmogsJNSipN3/IUftfPsm09nojJxkkQFmLGBWiuTwuQRgVUQE3CpNaqJLEBNQngnFprk
lqrTa3V1yQqcHR0dP3G+N6ffe+6040abQBZkpruFp/9/c9Q3SiLDMPApJSShEkmgBMgiVpKUd/9E/9q
mj/6/B8CaibqMAy11LJKJiqQRk1SSnn8iXeneeoL6wC73z8XZnzhX/+R5kA73z8XWme+sIFayqgZvL
Ox9+V5otP/fM0ak5QgUwANYn6+BPvTvPUF/5YTQKoSYBaayml1poT1CRqKUUFMnFSSqm1JgHUTJwAmT
hJU2tV6hAerXWal7F5Morxx/86Z9Ktz3UbAPIktR030et9Q8+85nvXruyXq+BYRiAJKU0oJSiJgGSA
Oq/8YGfTv07n/qdTIDMAZIAahrgfe//qTRPfvbTapL3vu8vZs6Tn/10EkD9yfe+P82Tn/10EkAF1FLK
e/6196b5/Sc/A6iZqEAmKpBGzS1+8r3vT/PkZz+dBmgt1ExUQAUYUfP9qUmATNRMADWvByRRk5j4moz
jzVK9ds/Fb7/nfX813f83qOn2F5BGTQPkVGoaII2aCZA5auYAadRMgDRqGiCnUtMAadTsMiBLUrPLUN
PtLyCNmgbIqdQ0QBo1EyBz1MwB0qiZAGnUNEBOpaYB0qjZZUCWpGaXoabbX0AaNQ2QU6lpgDRqJkDmq
JkDpFEzAdKoaYCCSk0DpFGzy4AsSc0uQ023v4A0ahogp1LTAGnUnACkUdMAadScAKRR0wA5LZoGSKNm
lwFZkppdhppufwFp1DRATqWmAdKoOQFIo6YB0qg5AUijpgFyKjUnkEbNLG0yJDW7DDXd/gLSqGmAnEp
NA6RRcwKQRk0DpFFzApBGTQPkVGoaII2aXQZkSwp2GWq6/QWkUdMAOZWaBkij5gQgjZoGSKPmBCCNmg
bIqdQ0QBo1uwzIktTsMtR0+wtIo6YBcio1DZBGzQRIo+Z2gDRqJkAaNQ2QU6lpgDRqdhmQJanZZajZB
pAlqenODpBGTQPkVGoaII2aCZBGze0AadRMgDRqGiCnUtMAadR0+ws12wCyJDXd2QHSqGmAnEpNA6RR
MwHSqLkdII2aCZBGTQPkVGoaII2abn+hZhtAlqSmOztAGjUnkFOpaYA0aiZAGjW3A6RRMwHSqGmAnEp
NA6RR0+0v1GwDyJLUdGcHskPmrgBp1EyANGrmaJmjZgKkUXNXgDRquv2Fmm0AWZKa7uwAadTcFSCNm
mQRs0cIHPUTIA0au4KkeZnt79Qsw0gS1LTnR0gjZq7AqRRMwHSqJkDZI6aCZBGzV0B0qjp9hdqtgFkS
Wq6sw0kUXNXgDRqJkAaNXOAzFEzAdKouStAGjXd/kLNNoAsSU13doCcKTUTII2aBkijpgHSqJkA0VNq
uv2Fmm5/ATLTaiZAGjUnkeZNA6RRMwFyptTsMiBLUrPLUNPtLyBnSs0ESK0mAdKoaYA0aiZazpSaXQZ
kSwp2GWq6/QXkTKmZAGnUNEaAaNQ2QRs0EyJLs8uALEnNLkNN120PyN1S080BsiQ1uw1Xbc9IHdLTT
cHyJLU7DLUDn32gNwtNd0cIEtSs8tQ0+0vICeouQWQiZo5QE5Q0wCZo+Z2gJxKTQPkBDX7AsiS10wy1
HT7C8gcNRMgjZo5QE5Q0wCZo+ZUQG5HTQPkBDX7AsiS10wy1GwDyJLUdGcHyBw1EyCNmjLATLDTAJmj
5lRABkdNA+QEnd0bA2q2AWRJArqzA2S0mgmQRs0cICeoaYDMUXMqiLejpgFygprujQE12wCyJDXd2QH
SqLlJQBo1DZAT1DRAGjV3DEijpgHSqDkBSK0m21+o2QaQJanpzg6QRs0dA9KoaYCCoKYB0qi5Y0AaNQ
2QRs0JQBo13f5CzTaALElNd3aANGruGJBGTQPkBDUnkEbNHQPSqGmANGpOANKo6fYXarYBZEIqurMDp
FFzx4A0ahogJ6hpgDRq7hiQRk0DpFFzApBGTbe/ULMNIEtS050dII2aBsgJahogp1LTAJmjpgFyx9Sc
CkijpntjQM02gCxJTXd2gDRqGiAnqGmAnEpNA2S0mgbiHVNzKiCnmu6NATXd/gLSqGmAnKCMAXiQnQ2
QOWoaIHdMzamANGr2BZAlqdllqOn2F5BGTQPkBDUNKFOpaYDMUdMAuWNqTgWkUbMvgCxJzS5DTbe/gM
xRMwHSqLkdIKdS0wBp1JwApFHTAGnUNEAmaVYrkCwp2Wwo6fYXkdLqJkAanbcD5FRqGiCnNmH0ANGoaI
I2aBshEzT4CsiQ1uw13f4CMkfNBEij5naAnEpNA6RRcwKQRk0DpFHTAJmo2UdAlqRml6Gm219A5qiZ
AGnU3A6QU6lpgDRqTgDSqGmANGoaIBM1+wjIktTsMtR0bwxAGjV3DMgdUzMHSKNmAqRR0wCZo2YCZI6
aXQZkSwp2GWq6NwYgjZo7BuS0qZkDpFEzAdKoaYDMUTMBmkfNLgOyJDW7DDXBALiKnd0ygdRq7hiQ06

ZmDpBGzQRIO6YBMkfNBMcNd3+Qs02gCxJTbcMIBM12wByx9TMATJR0wBp1DRA5qiZAJmjpttfqNkGk
CwP6XYBkEbNHCCNmhOANGoaII2aBshETQ0k3t2PMgAABxxJREFUUDPtL9RsA8iS1HS7AEijZg6QRs0J
QBo1DZBGTQNkoqYB0qjp9hdqgtgFkSWq6XQCkUTMHSPmBCCNmgiZIo6YBMlHTAGnUdPsLNdsAsiQ13dk
BcqBUTIDMUXM7Q06KmgmQOWq6/YWabQBZkpRU7AA5U2omQ0aouR0gd0XNBMcNd3+Qs02gCxJTXd2gJ
wpNRMgc9TcDpC7omYCZI6abn+hpttfQM6UmgmQOWpuB8hdUTMBMkfNLgOyJDW7DDXd/gLSqLkrQBo1J
wC5K2rmAGnUNEAmaVYRkCWp2Wwo6fYXkEbNXQHSqDkByF1RMwdIo6YBMlGzj4AsSc0uQ023v4A0au4K
kEbNCUDuipo5QBo1DZCJmn0EZEldhLquv0FpFFzV4A0ak4AcLfUzAHSqGmATNTsIyBLUrPLUNPtLyC
NmgbIqdQ0QBo1JwBp1DRAGjUNKBPU3DEgjZp9AWRJJanYZarr9BaRR0wA5LZoGSKPmBCCNmgiZIo6YBco
KaOwakUbMvgCxJzS5DTbe/gDRqGiCnUtMAadScAKRR0wBp1DRATLBzx4A0avYFkCWp2Wwo2QaQJanpz
g6QRk0D5FRqGiCnNhOANGoaII2aBsgJau4YkEZN98aAmm0AWZKa7uwAadQ0QE6lpgHSqJkA2YaaUwGZ
o6YBcio13f5CzTaALElNd3aANGoaIKdS0wBp1EyAbEPNqYDMUDMAOZWabn+hZhtAlqSmOztAGjUnkFO
paYA0aiZAtqHmVEDmqGmAnEpNt79Qsw0gS1LTnR0gjZoGyKnUNEAaANQ2Q06bmVEDmqGmAnEpNt79Qsw
0gS1LTnR0gjZoGyKnUNEAaANScAadTMAdKouStAJmoaII2abn+hZhtAlqSmOztAGjUnkFOpaYA0ak4A0
qiZA6RRc1eATNQ0QBo13f5CzTaALElNd3aANGoaIKdS0wBp1JwApFEzB0ij5q4AmahpgDRquv2Fmm5/
AWnU3BUgjZoGyAlqbgfICWrmADLBzT4CsiQ1uww13f4C0qi5K0AaNRMc9TcDpAT1MwBcoKafQRkSWp
2GWq6/QWkUXNXgDRqJkDmqLkdICEomQPkBDX7CMiS10wy1HT7C0ij5q4AadRMgMxRczATLAzB8gJav
YRkCWp2Wwo6fYXkD0LzGJkjpGyBw1JwC5K2r2BZAlqdlLqOn2F5AzpWYCZI6aBsgcNScAuStq9gWQJ
anZZajp9heQM6VmAmS0mgIHdUnALkravYFkCWp2Wwo6fYXkD0LzGJkjpGyBw1JwC5K2r2BZAlqdlL
qNkGkCWp6bqum40abQBZkpqu67o5qNkGkCWp6bqum40abQBZkpqu67o5qNkGkCWp6bqum40abQBZkpq
u67o5qNkGkCWp6bqum40abQBZkpqu67o5q0m67nwAsiQ1uww1XdedD0CWpGaXoabruvMByJLU7DLUDF
13PgBZkppdhpqu684HIEtSs8tQ03Xd+QBkSWp2GWq6rjsfgCxJzS5DTdd15w0QJanZZajZBpAlqem6r
puDmm0AWZKaruu60ajZBpAlqem6rpuDmm0AWZKaruu60ajZBpAlqem6rpuDmm0AWZKaruu60ajZBpAl
qem6rpuDmm0AWZKaruu60ajpuu58ALIkNbsMNV3XnQ9AlqRml6Gm67rzAcis10wy1HRddz4AWZKaXYa
aruv0ByBLUrPLUNN13fkAZEldhLquq47H4AsSc0uQ03XdecDkCWp2Wwo2QaQJanpuq6bg5ptAFmSmq
7rujmo2QaQJanpuq6bg5ptAFmSmq7rujmo2QaQJanpuq6bg5ptAFmSmq7rujmo2QaQJanpuq6bg5ptA
FmSmq7rujmo6brufACyJDW7DDVd150PQJakZpehpuu68wHIktTsMtR0XXc+AFmSmL2Gmq7rzgCGS1Kz
y1DTdd35AGRJanYZarqu0x+ALEnNLkNN13XnA5AlqdlLqNkGkCWp6bqum40abQBZkpqu67o5qNkGkCW
p6bqum40abQBZkpqu67o5qNkGkCWp6bqum40abQBZkpqu67o5qNkGkCWp6bqum40abQBZkpqu67o5q0
m67nwAsiQ1uww1XdedD0CWpGaXoabruvMByJLU7DLUDF13PgBZkppdhpqu684HIEtSs8tQ03Xd+QBkS
Wp2GWq6rjsfgCxJzS5DTdd15w0QJanZZajZBpAlqem6rpuDmm0AWZKaruu60ajZBpAlqem6rpuDmm0A
WZKaruu60ajZBpAlqem6rpuDmm0AWZKaruu60ajZBpAlqem6rpuDmm0AWZKaruu60ajpuu58ALIkNbs
MNV3XnQ9AlqRml6Gm67rzAcis10wy1HRddz4AWZKaXYaaruv0ByBLUrPLUNN13fkAZEldhLquq47H4
AsSc0uQ03XdecDkCWp2WX/N3y+mOLHwCY+AAAAEFTkSuQmCC\",\\n\\t\\\"errorCode\\\" :
0,\\n\\t\\\"errorInfo\\\" : \\\"No error!\\\"\\n}\\n\",

7 "result": 0

8 }

9 }

10

11

12

13 /**

14 * 生成打印预览图像

15 * 通过API调用生成打印预览图像

16 * @param printData - 打印数据对象

17 */

18 public async handlePreview(printData: any): Promise<void> {

19 if (!this.printSocketOpen || !this.nMPrintSocket) {

```

20         return alert("打印服务未开启");
21     }
22     if (!this.initBool) {
23         return alert("SDK未初始化");
24     }
25
26     try {
27         const initCanvasRes = await
this.initCanvas(printData.InitDrawingBoardParam)
28         if (!initCanvasRes) {
29             alert("初始化画布失败");
30             return;
31         }
32         const elementsProcessed = await
this.processPrintElements(printData.elements);
33         if (!elementsProcessed) {
34             alert("处理打印预览元素失败");
35             return;
36         }
37
38         const previewRes = await
this.nMPrintSocket.generateImagePreviewImage(8);
39         if (previewRes.resultAck.errorCode === 0 && previewRes.resultAck.info)
{
40             const imageData = JSON.parse(previewRes.resultAck.info).ImageData
41             this.previewImage = "data:image/png;base64," + imageData;
42             // alert("预览图已生成");
43             console.log("预览图已生成");
44         } else {
45             this.previewImage = null;
46             alert("获取预览图失败");
47         }
48     } catch (err) {
49         console.error(err);
50         this.previewImage = null;
51         alert("打印预览异常");
52     }
53     this._updateReactState();
54 }

```

四、打印接口说明

4.1 设置打印回调

代码块

```

1  export default class NMPrintSocket {
2      // 添加打印回调
3      public addPrintListener(callback: (msg: any) => void): (msg: any) => void
4  }

```

代码块

```

1  let printListener = null;
2
3  printListener = this.nMPrintSocket.addPrintListener(async (msg) => {
4      const resultAck = msg?.resultAck;
5
6      if (resultAck?.errorCode === 0 && resultAck?.info === "commitJob ok!") {
7          await strategyFactory.handleCommitSuccess();
8      }
9      //已接入历史版本客户仍可以使用printQuantity和onPrintPageCompleted字段获取打印进度
10
11     if (resultAck?.printCopies != null && resultAck?.printPages != null) {
12         strategyFactory.handleProgressUpdate(resultAck);
13     }
14
15     if (resultAck?.printCopies === printQuantity &&
16         resultAck?.printPages === list.length) {
17         await strategyFactory.handleCompletion();
18     }
19
20     if (resultAck?.errorCode !== 0) {
21         strategyFactory.handleError(msg);
22     }
23 });

```

4.2 移除打印回调

代码块

```

1  export default class NMPrintSocket {
2      // 移除打印回调
3      public removePrintListener(callback: (msg: any) => void): void
4  }

```

代码块

```

1  const cleanupListener = () => {
2      if (printListener && this.nMPrintSocket) {
3          this.nMPrintSocket.removePrintListener(printListener);

```

```

4         printListener = null;
5     }
6 };

```

4.3 开始打印

代码块

```

1  export default class NMPrintSocket {
2      /**
3       * 开始一个打印任务。
4       *
5       * @param {number} printDensity - 打印浓度，根据不同打印机型号取值范围不同，具体
        如下：
6       *
7       *                                     - B3S、B203、B1、K3、K3W、M2：取值范围
        1~5，默认为 3。
8       *                                     - B50、B11、B50W、B32、Z401：取值范围
        1~15，默认为 8。
9       * @param {number} printLabelType - 纸张类型，可选值：
10      *                                     1：间隙纸
11      *                                     2：黑标纸
12      *                                     3：连续纸
13      *                                     4：定孔纸
14      *                                     5：透明纸
15      *                                     6：标牌
16      *                                     10：黑标间隙纸
17      * @param {number} printMode - 打印模式，可选值：
18      *                                     1：热敏
19      *                                     2：热转印
20      *                                     注意，不同打印机型号支持的打印模式有限制，具体如下：
        - D11、D101、D110、H10、B16、B18、B3S、B203、
        B1、K3、K3W、B11 仅支持热敏。
21      *                                     - B50、B50W、B32、Z401、M2 仅支持热转印。
22      * @param {number} count - 总打印份数，表示所有页面的打印份数之和。
23      *                                     例如，如果你有3页需要打印，第一页打印3份，第二页打
        印2份，第三页打印5份，那么count的值应为10（3+2+5）。
24      * @return {Promise} - 返回一个 Promise，解析为开始打印任务的结果
25      * @example
26      * //返回数据示例
27      * {
28      *     "apiName": "startJob",
29      *     "resultAck": {
30      *         "errorCode": 0,
31      *         "info": "startJob ok!",
32      *         "result": 0
33      *     }

```

```

34     * }
35     * @description 返回结果中的 errorCode 含义如下:
36     *             - 0: 成功
37     *             - -1: 失败, info 表示原因
38     *             - -2: 打印机忙碌, info 表示原因
39     *             - -3: 打印机接收到不支持的参数, 主要是浓度、纸张类型、打印模式,
    info 表示具体原因
40     */
41     public startJob(
42         printDensity: number,
43         printLabelType: number,
44         printMode: number,
45         count: number
46     ): Promise<any>
47 }

```

代码块

```

1  //返回数据示例
2  {
3      "apiName": "startJob",
4      "resultAck": {
5          "errorCode": 0,
6          "info": "startJob ok!",
7          "result": 0
8      }
9  }
10
11  const startRes = await this.nMPrintSocket.startJob(
12      this.density,
13      this.label_type,
14      this.print_mode,
15      list.length * printQuantity
16  );
17
18  if (startRes.resultAck.errorCode !== 0) {
19      cleanupListener();
20  }

```

4.4 提交打印任务

代码块

```

1  export default class NMPrintSocket {
2      /**
3      * 提交一个打印任务。

```

```

4      *
5      * @param {string} [printData=null] - 打印数据的 JSON 字符串。
6      * @param {string} printerImageProcessingInfo - 打印机图像处理信息的 JSON 字符串，包含打印份数信息，格式如下：
7      * {
8      *   "printerImageProcessingInfo": {
9      *     "printQuantity": 1 // 用于指定当前页的打印份数。例如，如果需要打印3页，第一
        页打印3份，第二页打印2份，第三页打印5份，则在3次提交数据时，
        printerImageProcessingInfo 中的 "printQuantity" 值分别应为 3, 2, 5。
10     *   }
11     * }
12     * @return {Promise} - 返回一个 Promise，解析为提交打印任务的结果
13     */
14     public commitJob(printData: string | undefined,
        printerImageProcessingInfo: string): Promise<any>
15 }

```

代码块

```

1  //数据提交成功返回数据示例
2  {
3      "apiName": "commitJob",
4      "resultAck": {
5          "errorCode": 0,
6          "info": "commitJob ok!",
7          "result": 0
8      }
9  }
10
11 //打印进度返回示例1：此回调的含义为第一页第一份打印完成
12 {
13     "apiName": "commitJob",
14     "resultAck": {
15         "errorCode": 0,
16         "info": "",
17         "onPrintEPCCodeCompleted": "",
18         "onPrintPageCompleted": 1, //打印完成份数回调
19         "onPrintPageLengthCompleted": "38.00",
20         "printQuantity": 1 //打印完成页数回调
21     }
22 }
23
24 //打印进度返回示例1：此回调的含义为第一页第二份打印完成
25 {
26     "apiName": "commitJob",
27     "resultAck": {

```

```

28         "errorCode": 0,
29         "info": "",
30         "onPrintEPCCodeCompleted": "",
31         "onPrintPageCompleted": 2, //打印完成份数回调
32         "onPrintPageLengthCompleted": "38.00",
33         "printQuantity": 1 //打印完成页数回调
34     }
35 }
36
37 //打印进度返回示例1: 此回调的含义为第二页第一份打印完成
38 {
39     "apiName": "commitJob",
40     "resultAck": {
41         "errorCode": 0,
42         "info": "",
43         "onPrintEPCCodeCompleted": "",
44         "onPrintPageCompleted": 1, //打印完成份数回调
45         "onPrintPageLengthCompleted": "38.00",
46         "printQuantity": 2 //打印完成页数回调
47     }
48 }
49
50
51 public async commitPrintJob() {
52     if (!this.printSocketOpen || !this.nMPrintSocket) {
53         return alert("打印服务未开启");
54     }
55     this.nMPrintSocket.commitJob(undefined, JSON.stringify(this.jsonObj));
56 }

```

4.5 结束打印任务

代码块

```

1  export default class NMPrintSocket {
2      /**
3       * 结束一个打印任务。
4       *
5       * @return {Promise} - 返回一个 Promise, 解析为结束打印任务的结果
6       */
7      public endJob(): Promise<any>
8  }

```

代码块

```

1  //返回数据示例

```

```

2  {
3      "apiName": "endJob",
4      "resultAck": {
5          "errorCode": 0,
6          "info": "endJob ok!",
7          "result": 0
8      }
9  }
10
11  if (!this.nMPrintSocket) return;
12  const endRes = await this.nMPrintSocket.endJob();
13  if (endRes.resultAck.errorCode === 0) {
14      console.log("打印完成");
15  }
16
17  cleanupListener();

```

4.6 取消打印任务

代码块

```

1  export default class NMPrintSocket {
2      /**
3       * 取消当前的打印任务，并执行回调函数。
4       *
5       * @return {Promise} 返回一个 Promise，解析为取消打印任务的结果
6       */
7      public cancelJob(): Promise<any>
8  }

```

代码块

```

1  try {
2      const cancelJobRes = await this.nMPrintSocket.cancelJob();
3      if (cancelJobRes.resultAck.errorCode == 0) {
4          console.log("取消打印成功");
5      }
6  } catch (err) {
7      console.error(err);
8  }

```

五、回调说明

代码块


```

1
2  /**
3   * {
4   *   "apiName": string, // 调用的 API 名称
5   *   "resultAck": {
6   *     "errorCode": number, // 错误代码, 0 表示成功, 其他值表示错误
7   *     "info": string, // 信息字符串, 描述操作结果
8   *     "result": number // 结果代码, 通常与 errorCode 一致
9   *   }
10  * }
11  */
12  {
13    "apiName": "commitJob",
14    "resultAck": {
15      "errorCode": 0,
16      "info": "commitJob ok!",
17      "result": 0
18    }
19  }

```

六、错误码相关说明

6.1 错误码说明描述

代码块

```

1  * 0-无错误
2  //打印机返回部分
3  * 1-盒子打开
4  * 2-缺纸
5  * 3-电量不足
6  * 4-电池异常
7  * 5-手动停止
8  * 6-数据错误
9  * 7-温度过高
10 * 8-走纸异常
11 * 9-正在打印
12 * 10-未检测到打印头
13 * 11-环境温度过低
14 * 12-打印头松动
15 * 13-未检测到碳带
16 * 14-不匹配的耗材
17 * 15-用完的碳带
18 * 16-不支持的纸张类型
19 * 17-设置纸张类型失败
20 * 18-设置打印模式失败

```

```
21 * 19-设置浓度失败
22 * 20-写入rfid失败
23 * 21-边距参数错误
24 * 22-超时错误
25 * 23-断开连接
26 * 24-画板参数设置错误
27 * 25-旋转角度参数错误
28 * 26-json参数错误
29 * 27-出纸异常（关闭上盖检测）
30 * 28-检查纸张类型
31 * 29-碳带与打印模式不匹配
32 * 30-设置浓度不支持
33 * 31-不支持的打印模式
34 * 32-标签材质设置异常，请重新设置
35 * 33-不支持该标签材质，请更换或重新设置
36 * 34-不支持RFID写入
37 * 50-非法标签
38 * 51-非法碳带和标签
39
40 //内部使用
41 //E_UNKNOW_ERROR = 255,
```

